



ALMA MATER STUDIORUM UNIVERSITY OF BOLOGNA
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING – DISI

Back To The Future: xLSTM Outperforms Large Language Models

Text Pre-processing, Language Models, Recurrent Neural Networks, Seq2Seq, Attention Mechanisms and xLSTM

Prof. Gianluca Moro

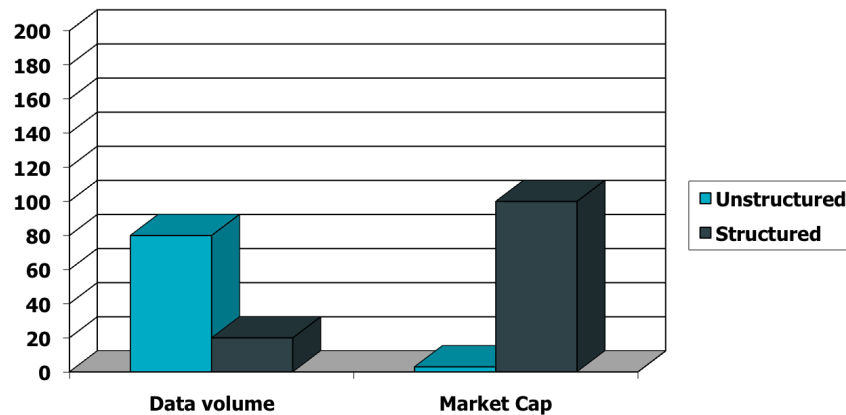
DISI, University of Bologna
Via dell'Università, 50 I-47522 Cesena (FC), Italy
{name.surname}@unibo.it



Prelude to Transformers - Moro G., Frisoni G.

1

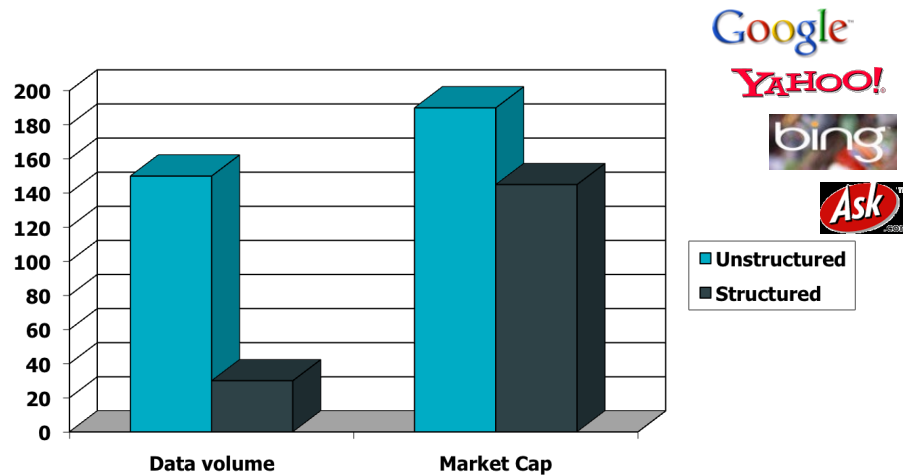
Unstructured vs. structured data in 1996



Prelude to Transformers - Moro G., Frisoni G.

2

Unstructured vs. structured data in 2009



Prelude to Transformers - Moro G., Frisoni G.

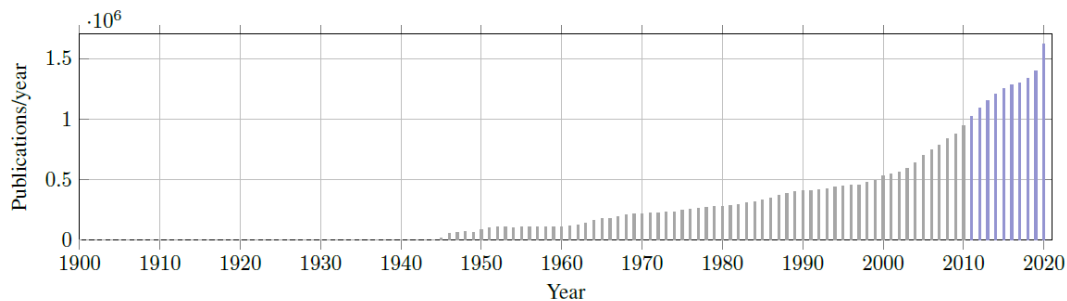
3

The Increasing Importance of unstructured data (ii)

- “85% of all data stored is held in an unstructured format”
Butler Group
- “80% of business is conducted on unstructured information”
Gartner Group
- “Unstructured data **doubles** every three months”
Gartner Group

Example: Exponential Growth of Papers in Healthcare

- Growing availability of unstructured textual content on the Web
 - Human impossibility to keep up with the accumulation of information in text documents
 - E.g. More than 1 million new scientific papers added to the PubMed database each year
 - From 2020, about **3 papers per minute!**



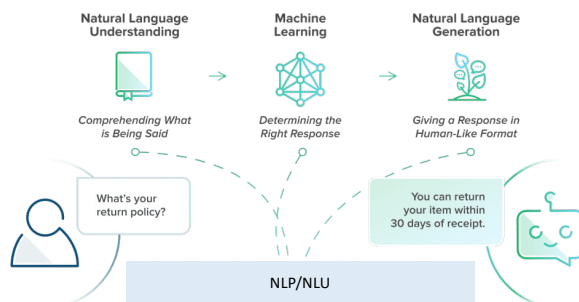
disi-unibo-nlp.github.io

Prelude to Transformers - Moro G.,
Frisoni G.

5

Example: NLP/NLU in Insurance Market – i

- The global NLP/NLU market is expected to be worth \$35 billion by 2025 [1]
- In 2019 [2], 84% of insurers said they believed AI will revolutionize their sector
- Accenture's Living Business survey [3]
 - 64% of customer churns is for more relevant product, service or experience in the new insurer
 - AI solutions are essential to provide customers with the experience they're expecting



[1] <https://www.mordorintelligence.com/industry-reports/natural-language-processing-market>

[2] <https://www.statista.com/statistics/1142780/insurance-players-opinion-impact-ai-insurance-sector-france/>

[3] <https://insuranceblog.accenture.com/exploring-artificial-intelligence-for-insurance/>

disi-unibo-nlp.github.io

Prelude to Transformers - Moro G.,
Frisoni G.

6

Example: NLP/NLU in Insurance Market – ii

- Customer service and assistance
 - Experience Personalization
 - >80% of insurance customers want personalized offers, messages, pricing and recommendations from their car, home and life insurance providers [4]
 - Automatic Question Answering
 - Provide customers with timely and accurate answers to their questions
 - An insurance agent can waste a considerable amount of time on low added-value tasks like answering basic questions
 - IBM has estimated that within a 6-minute customer service call, agents spend 75% of that time doing manual research to find the relevant information [5]
- Claims management
 - Claims processing is a core activity within the insurance sector that is at the center of many pain points. The task can be time-consuming, costly and subject to human errors
 - E.g., Insurance agents can use NLP/NLU during phone calls to recognize a client's speech and automatically fill out a claims form. Employees then simply manually verify the results.

[4] https://www.accenture.com/_acnmedia/Accenture/Conversion-Assets/Secure/pdf-no-index-3/Accenture-Satisfying-the-Craving-for-Insurance-Personalisation

[5] <https://www.ibm.com/blogs/watson/2017/10/10-reasons-ai-powered-automated-customer-service-future/>

[disi-unibo-nlp.github.io](https://github.com/disi-unibo-nlp)

Prelude to Transformers - Moro G., Frisoni G.

7

Example: NLP/NLU in Insurance Market – iii

- Fraud Detection
 - A large variety of traditional and experimental technologies have been leveraged in combatting **insurance fraud** [6]
 - Experts consider NLP/NLU technologies among the most promising options in the field
 - Manually **reviewing claims** notes and data like emails, social media posts, or text messages can be incredibly **time-consuming**; NLP solutions are able to **automatically flag cases of suspected fraud** for human review
- Underwriting automation
 - **Underwriters** have to analyze many policies and documents to **make key decisions**
 - NLP/NLU solutions for underwriting use cases can **extract relevant information** to help underwriters **assess risk levels** (dates, locations, names, diagnoses, lab results, etc.)
 - These solutions help underwriters **access information** they would have taken **hours to find** by parsing through documentation themselves

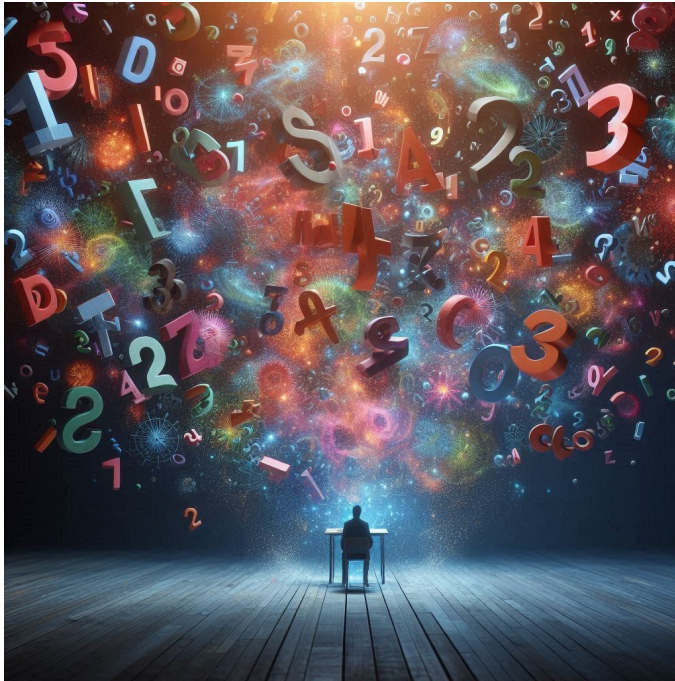
[6] <https://earlymetrics.com/combating-insurance-fraud-with-tech-vicious-cycle/>

[disi-unibo-nlp.github.io](https://github.com/disi-unibo-nlp)

Prelude to Transformers - Moro G., Frisoni

G.

8



Tokenization: A Necessary Premise

Prompt: Letters transforming into numbers

Generated with Microsoft Copilot Designer

Text Tokenization: a Mere Syntactic Operation ?

- A **token** is a sequence of chars followed by a delimiter, which is one or more chars, usually the delimiter is the space ...
 - **Text:** *"Friends, Romans, Countrymen lend me your ears!"*
 - **Token list:** *Friends Romans Countrymen lend me your ears*
- ... but the space is not a solution, example:
 - *Finland's capital* → *Finland ? Finlands ? Finland's ?*
 - *Hewlett-Packard* → *Hewlett* and *Packard* 2 tokens ?
 - *state-of-the-art co-author* dividing always the terms ?
 - *lowercase, lower-case, lower case* several forms for the same concept *San Francisco*: 1 or 2 tokens ?
 - date *3/12/91 Mar. 12, 1991 12/3/91 55 A.C.*
 - num. and codes with spaces *(800) 234-2333 connection error 25401*
- "Morgen will ich in MIT" here MIT is German ? *Language identification is required*
- In general it is required a semantic understanding - It is not a syntactic operation

Tokenization in Other Language: Further Problems

- French
 - *L'ensemble* → 1 or 2 tokens ?
 - *L ? L' ? Le ?* it should match with *un ensemble*
 - until 2003 Google did not perform this match
- German: long composed nouns without separations
 - *Lebensversicherungsgesellschaftsangestellter*
 - 'employee of a life insurance company'
 - the efficacy of answers increases of 15% applying splitting techniques
- Chinese and Japanese don't have space among words
 - 莎拉波娃现在居住在美国东南部的佛罗里达。
 - it's difficult to guarantee an unique tokenization
- Arabic and Hebrew are written from right to left, but sometimes is the opposite, for instance with numbers

استقلت الجزائر في سنة 1962 بعد 132 عام من الاحتلال الفرنسي.

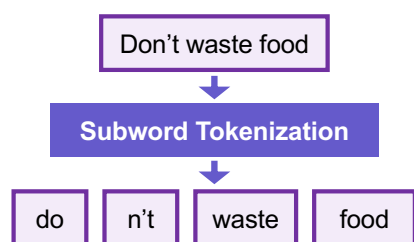
[disi-unibo-nlp.github.io](https://github.com/disi-unibo-nlp)

Prelude to Transformers - Moro G., Frisoni G.

11

Tokenization: How Neural Networks «See»

- Neural networks cannot work directly on text data
 - Since they work exclusively with vector data, text must be converted into numerical representations
- Tokenization is a crucial step in today's NLP
 - The process of **breaking down a piece of text into smaller indivisible units**, called **tokens**
 - Tokens can be **words**, **subwords** (n-gram characters), or **characters**
 - **Deep Learning architectures in NLP process the raw text at the token level**



Word	Token(s)
surf	['surf']
surfing	['surf', '##ing']
surfboarding	['surf', '##board', '##ing']
surfboard	['surf', '##board']
snowboard	['snow', '##board']
snowboarding	['snow', '##board', '##ing']
snowing	['snow', '##ing']

Tokenization is deeper than it seems

- E.g. each network model can better understand the semantic similarities between **surfboarding** and **snowboarding**, **snowing** and **snowboarding**, with subword tokenization

[disi-unibo-nlp.github.io](https://github.com/disi-unibo-nlp)

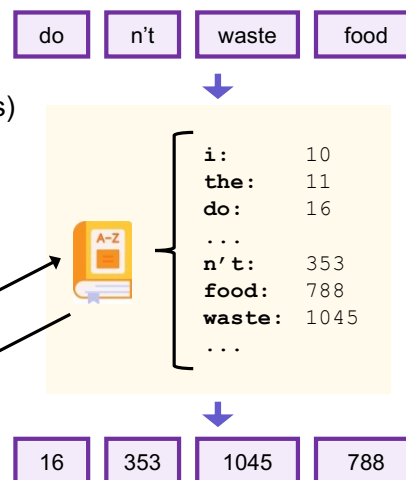
Prelude to Transformers - Moro G., Frisoni G.

12

Tokenization Algorithms and Vocabulary Mapping

Tokenizers are based on linguistic algorithms and partially learned

- **Whitespace** tokenization
- Punctuation-based tokenization
- Regular expression tokenization
- **Entity-based** tokenization (e.g., city, company names)
- **Subword tokenization**
 - E.g., some tokenizers iteratively merge the most frequent pairs of characters or subwords
- Character tokenization



Each language model has a predefined vocabulary of tokens, previously produced by tokenizers, on which it is trained on

Different Models, Different Tokenizers

```
1 # Display the results
2 print("Original Text:", sample_text)
```

Original Text: Tokenizer architectures for LLMs are fascinating!

```
1 print("\nBERT (WordPiece) Tokens:", bert_tokens)
2 print("\nGPT-2 (Byte-Pair Encoding) Tokens:", gpt2_tokens)
3 print("\nRoBERTa (Byte-level BPE) Tokens:", roberta_tokens)
4 print("\nT5 (SentencePiece) Tokens:", t5_tokens)
```

BERT (WordPiece) Tokens: ['token', '##izer', 'architecture', '##s', 'for', 'll', '##ms', 'are', 'fascinating', '!']

GPT-2 (Byte-Pair Encoding) Tokens: ['Token', 'izer', 'Ġarchitectures', 'Ġfor', 'ĠLL', 'Ms', 'Ġare', 'Ġfascinating', '!']

RoBERTa (Byte-level BPE) Tokens: ['Token', 'izer', 'Ġarchitectures', 'Ġfor', 'ĠLL', 'Ms', 'Ġare', 'Ġfascinating', '!']

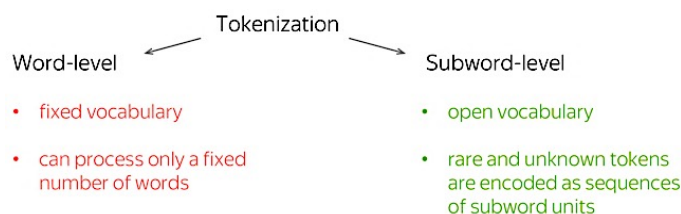
T5 (SentencePiece) Tokens: ['_To', 'ken', 'izer', '_architecture', 's', '_for', '_L', 'LM', 's', '_are', '_fascinating', '!']

« _ » indicates a space

«##» indicates that the token is a continuation of the previous token

Word-Level vs Subword-Level Tokenization

- The **fixed vocabulary problem**
 - Input tokens not in the vocabulary are replaced with a special **UNK** («unknown») token
 - If we use a straightforward word-level tokenization (i.e., our tokens are words), we will be able to process a fixed number of words
 - We will get lots of unknown tokens and our model will lose their meaning
- How can we represent all words, even those not seen in the training data ?
 - If you are not familiar with a word, you are familiar with the parts it consists of: **subwords** (in the worst case, symbols)
 - Then, why do not we split the rare and unknown words into smaller parts ?



Byte Pair Encoding

- Byte Pair Encoding (BPE) [[Gage, 1994](#)] is a **data compression technique** that iteratively **replaces the most frequent pairs of bytes with a single new byte**
- We refer to an **adaptation for subword segmentation (de-facto standard)**
 - Instead of merging frequent pairs of bytes, it **merges characters or character sequences**
 - It powers tokenization in all GPT models so far
- The modern BPE algorithm, after applying a **whitespace** tokenization to get a list of initial terms, consists of two parts
 - (1) **Training**: learn «BPE rules», i.e., which pairs of symbols to merge
 - (2) **Inference**: apply learned rules to segment a text



BPE iteratively merges the most frequent pairs of tokens in a text corpus until a predefined vocabulary size is reached

- **Easy to implement and efficient to run**
- It is a **lossless** algorithm: the **original text can be reconstructed** from the BPE-encoded tokens
- After training, it keeps **frequent words intact and splits rare/unknown** ones into small known parts

Byte Pair Encoding: Training Example – i

- Input, let C be the corpus "hug hug hug .. pug pug .. pun pun .. bun .. hugs hugs ..." and V the initial BPE vocabulary; **for simplicity we ignore the space character**
 - $C = [(\text{"hug"}, 10), (\text{"pug"}, 5), (\text{"pun"}, 12), (\text{"bun"}, 4), (\text{"hugs"}, 5)]$ # whitespace tokenization
 - $V = [\text{"b"}, \text{"g"}, \text{"h"}, \text{"n"}, \text{"p"}, \text{"s"}, \text{"u"}]$ # character occurrences in C
- The training starts by splitting each C word into V characters
 - $[(\text{"h"} \text{"u"} \text{"g"}, 10), (\text{"p"} \text{"u"} \text{"g"}, 5), (\text{"p"} \text{"u"} \text{"n"}, 12), (\text{"b"} \text{"u"} \text{"n"}, 4), (\text{"h"} \text{"u"} \text{"g"} \text{"s"}, 5)]$
- **Stage 1.** Most frequent pair: $(\text{"u"} \text{"g"})$, which occurs 20 times
 - Merge rule: $(\text{"u"} \text{"g"}) \rightarrow \text{"ug"}$
 - $V = [\text{"b"}, \text{"g"}, \text{"h"}, \text{"n"}, \text{"p"}, \text{"s"}, \text{"u"}, \text{"ug"}]$
 - $C = [(\text{"h"} \text{"ug"}, 10), (\text{"p"} \text{"ug"}, 5), (\text{"p"} \text{"u"} \text{"n"}, 12), (\text{"b"} \text{"u"} \text{"n"}, 4), (\text{"h"} \text{"ug"} \text{"s"}, 5)]$

Byte Pair Encoding: Training Example – ii

- **Stage 2.** Most frequent pair: $(\text{"u"} \text{"n"})$, which occurs 16 times
 - Merge rule: $(\text{"u"} \text{"n"}) \rightarrow \text{"un"}$
 - $V = [\text{"b"}, \text{"g"}, \text{"h"}, \text{"n"}, \text{"p"}, \text{"s"}, \text{"u"}, \text{"ug"}, \text{"un"}]$
 - $C = [(\text{"h"} \text{"ug"}, 10), (\text{"p"} \text{"ug"}, 5), (\text{"p"} \text{"un"}, 12), (\text{"b"} \text{"un"}, 4), (\text{"h"} \text{"ug"} \text{"s"}, 5)]$
- **Stage 3.** Most frequent pair, 15 times $(\text{"h"} \text{"ug"})$, our first three-letter token
 - Merge rule: $(\text{"h"} \text{"ug"}) \rightarrow \text{"hug"}$
 - $V = [\text{"b"}, \text{"g"}, \text{"h"}, \text{"n"}, \text{"p"}, \text{"s"}, \text{"u"}, \text{"ug"}, \text{"un"}, \text{"hug"}]$
 - $C = [(\text{"hug"}, 10), (\text{"p"} \text{"ug"}, 5), (\text{"p"} \text{"un"}, 12), (\text{"b"} \text{"un"}, 4), (\text{"hug"} \text{"s"}, 5)]$
- ... and the process continues until we reach the desired vocabulary size

Byte Pair Encoding: Inference

- Inference follows the training process closely applying the learned rules
- **Step 1.** Split the input words into individual characters
- **Step 2.** Iteratively apply the **highest merge rule in the table** (among all possible merges) on those splits, until no merge is possible
- **Note:** the merge table is ordered
 - Higher merges were more frequent in the training data (that's why they have priority)
 - The BPE segmentation is **deterministic**

("u" "g") → "ug"
("u" "n") → "un"
("h" "ug") → "hug"

Merge Table

"bug" → ["b" "ug"]
"mug" → ["[UNK]" "ug"]
"thug" → ["[UNK]" "hug"]

[UNK] indicates that certain character combinations or tokens in the input text did not appear in the training data

Byte Pair Encoding: Training

- The algorithm builds a **merge table** and a **vocabulary of tokens**
 - At the beginning, the merge table is empty and the vocabulary consists of characters
- The BPE algorithm can be expressed as a **recursive function**

C = corpus
Usually, Wikipedia dumps, Common Crawl, Books, Social media content

V = vocabulary
For real-world cases, the base vocabulary will contain at least all the ASCII characters and possibly some Unicode characters

Append the learned merge rule to the merge table

k = target vocabulary size;
Usually 30,000 – 100,000

```
def bpe(V, C, k):  
    # base case: if V reached target size  
    if len(V) >= k:  
        return V  
    # Find most frequent adjacent pair of tokens in C  
    # Only consider pairs where both tokens are in V  
    pair = argmax(f(a, b, C) for (a, b) in get_adjacent_pairs(C, V))  
    # Create a new token by merging the pair  
    new_token = merge(pair[0], pair[1])  
    # Keep both original tokens AND the new merged token  
    V = V + {new_token}  
    # Update C by replacing all occurrences of pair with new token  
    C = replace_all(C, pair, new_token)  
    # Recursive call with updated V and C  
    return bpe(V, C, k)
```

Tokenizer: Not A Technical Detail

- The tokenizer
 - It is partially trained separately from the language model
 - Essentially impossible to change it afterwards without retraining
- It may seem like a technical detail but it is foundational to how LLMs operate



Andrej Karpathy
Co-founder of OpenAI

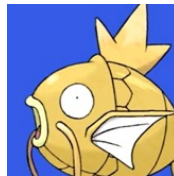
Tokenization is at the heart of much weirdness of LLMs. Do not brush it off.

- Why can't LLM spell words? **Tokenization.**
- Why can't LLM do super simple string processing tasks like reversing a string? **Tokenization.**
- Why is LLM worse at non-English languages (e.g. Japanese)? **Tokenization.**
- Why is LLM bad at simple arithmetic? **Tokenization.**
- Why did GPT-2 have more than necessary trouble coding in Python? **Tokenization.**
- Why did my LLM abruptly halt when it sees the string "<|endoftext|>"? **Tokenization.**
- What is this weird warning I get about a "trailing whitespace"? **Tokenization.**
- Why the LLM break if I ask it about "SolidGoldMagikarp"? **Tokenization.**
- Why should I prefer to use YAML over JSON with LLMs? **Tokenization.**
- Why is LLM not actually end-to-end language modeling? **Tokenization.**
- What is the real root of suffering? **Tokenization.**

Faulty Tokens Due to Noisy Training Data

- Many tokenizers have under-trained or faulty tokens
 - This is due to noisy training data such as unfiltered social net posts, e.g. Reddit forums
- A while back, people noticed some strange tokens in GPT-J (~GPT-2)

- « RandomRedditorWithNo»
- «davidjl»
- « SolidGoldMagikarp»
- «BuyableInstoreAndOnline»

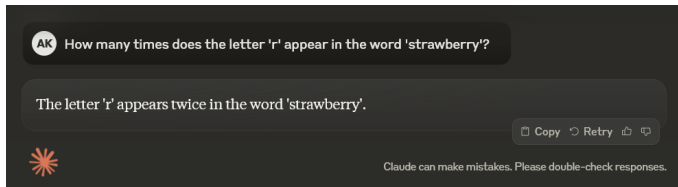


- StarCoder2 [[Lozhkov et al., 2024](#)] has a public dataset for tokenizer training
 - they mention «It contains our personal favourite token of all time» 🏆

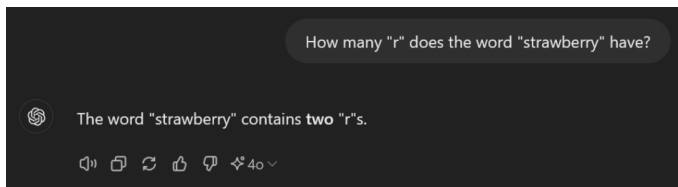
ittrLorem**ipum**dolorsitametconsecteturadipiscinelitIntegervelvel

The Strawberry Problem 🍓

- Counting «r» in «Strawberry»



Claude 3.5 Opus



GPT4o

Tokens	Characters
11	35
How many Rs in the word STRAWBERRY?	

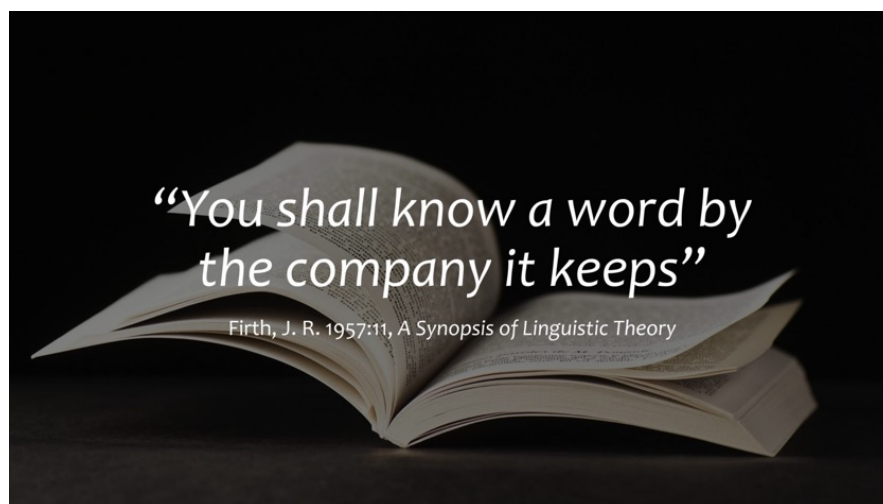


Language Modeling

Prompt: A robot walking in a multidimensional space made up of tokens

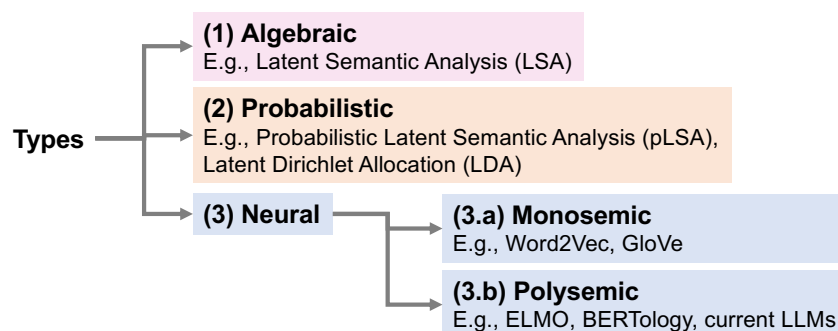
Generated with Microsoft Copilot Designer

Distributional Hypothesis



What is a Language Model?

- A language model (LM) is a type of AI model that **learns the statistical structure and patterns of a language to understand or generate it**
 - LMs rely on **latent spaces** built by **co-occurrences** of words
 - Vector components are the **latent dimensions**, whose number is an hyperparameter
 - The objective is to **make language semantics emerge** from textual data by learning **embeddings** (meaningful, dense representations)



Word2Vec CBOW: The simplest Neural LM

Input

- Context $C = \{w_{t-m}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+m}\}$

Output

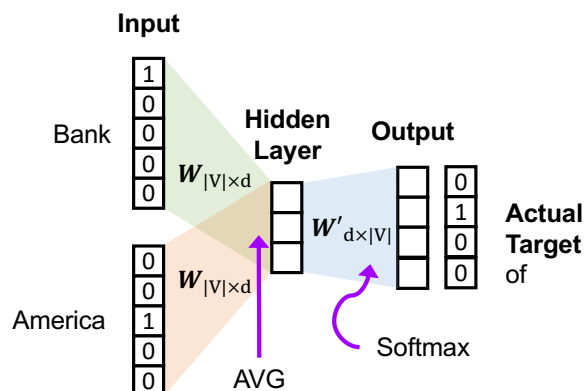
- Target word w_t

Embedding layer

- Each context word $w_i \in C$ is represented by a one-hot vector $x_i \in \mathbb{R}^{|V|}$, with V being the vocabulary
- These vectors are mapped to embeddings $h_i \in \mathbb{R}^d$ via a shared weight matrix W
- The hidden layer computes the average embedding $h = \frac{1}{2m} \sum_{i \in C} W x_i$

Output layer

- Probability distribution over V (softmax)



Training: Minimize the negative log-likelihood $-\log P_{\theta}(w_t|C)$

Word Embeddings are in the Hidden Layer

- In our example, suppose we're learning word vectors with 300 features and a vocabulary of 10000 words
- the hidden layer h is a weight matrix W with 10,000 rows (one for every word in our vocabulary) and 300 columns (one for every hidden neuron)
 - 300 features are used in the published model trained on the Google news dataset
 - 300 is a *hyper parameter* to be tuned

- The rows of this weight matrix are actually what will be our word vectors!

- So the end goal is really just to learn this hidden layer weight matrix

- The second matrix W' served just for the training loss

$$h = \frac{1}{C} W^T (x_1 + x_2 + \dots + x_C)$$

$$= \frac{1}{C} (v_{w_1} + v_{w_2} + \dots + v_{w_C})^T$$

hidden layer



$$E = -\log p(w_O | w_{I,1}, \dots, w_{I,C})$$

$$= -u_j + \log \sum_{j'=1}^V \exp(u_{j'})$$

$$= -v'_{w_O} \cdot h + \log \sum_{j'=1}^V \exp(v'_{w_j} \cdot h)$$

u_j = logit of the word to predict

v'_{w_O} word to predict,
 v'_{w_j} j-th column of W'

Word Embeddings: Mathematical Steps for the Loss

$$E = -\log p(w_0 | w_{I,1}, \dots, w_{I,C})$$

$$p(w_0 | w_{I,1}, \dots, w_{I,C}) = \frac{\exp(u_{w_0})}{\sum_{j=1}^V \exp(u_j)}$$

$$E = -\log \left(\frac{\exp(u_{w_0})}{\sum_{j=1}^V \exp(u_j)} \right)$$

$$E = - \left(\log(\exp(u_{w_0})) - \log \left(\sum_{j=1}^V \exp(u_j) \right) \right)$$

$$E = -u_{w_0} + \log \left(\sum_{j=1}^V \exp(u_j) \right)$$

Hidden Layer Weight Matrix $\rightarrow$ Word Vector Lookup Table!

300 neurons $\rightarrow$ 300 features

10,000 words $\rightarrow$ 10,000 words

loss

$$E = -\log p(w_0 | w_{I,1}, \dots, w_{I,C})$$

u_j : logit of the word w_0 to predict

$$= -u_{j^*} + \log \sum_{j=1}^V \exp(u_j)$$

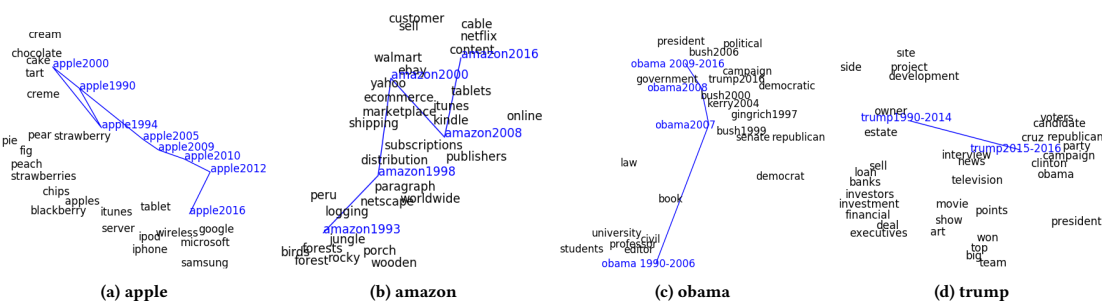
$$= -\mathbf{v}'_{w_0} \cdot \mathbf{h} + \log \sum_{j=1}^V \exp(\mathbf{v}'_{w_j} \cdot \mathbf{h})$$

$\mathbf{v}'_{w_0}$ word to predict, $\mathbf{v}'_{w_j}$ j-th column of $\mathbf{W}'$

disi-unibo-nlp.github.io Prelude to Transformers - Moro G., Frisoni G. 32

How the Meaning of Apple, Amazon, Obama, Trump Moved Over Time

- word2vec applied to 99,872 articles of The New York Times between 1990 and 2016 and projected in 2-D with t-SNE



- Apple** trajectory: from fruit and dessert to iPhone, iPad until early 2000s with the recovery by Steve Jobs
- Amazon** shifted from a forest to an e-commerce company, landing in 2016 as an online-streaming provider for the popularity of its Prime Video service
- Obama** from a civil rights attorney and university professor to political sphere, as **Trump** from a real estate developer and TV celebrity

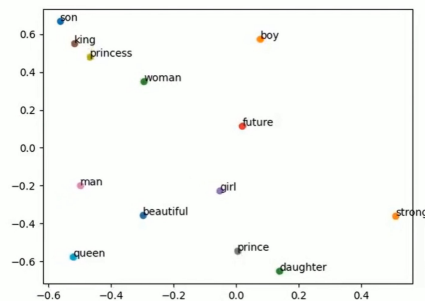
Word2Vec CBOW Embeddings

- For example, from a large number of sentences such as:
 - The future **king** is the **prince**
 - Daughter is the **princess**
 - Son is the **prince**
 - Only a **woman** can be a **queen**
 - The **princess** is a **woman**
 - The **princess** will be a **queen**
 - Queen** and **king** rule the team
- We get word embeddings such as:
 - queen** = $(-0.10, -0.02, 0.04, \dots)$
 - princess** = $(-0.05, -0.01, 0.02, \dots)$
 - prince** = $(-0.35, 0.15, -0.12, \dots)$
- The vector learned for **queen** is closer to that of **princess** than to **prince** as the first two vector contexts share more terms, e.g., *woman*

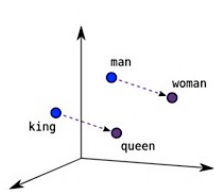
Visualization of 2D embeddings learned by a neural net with a single hidden layer

(Word2Vec CBOW, 1700 epochs, royal data)

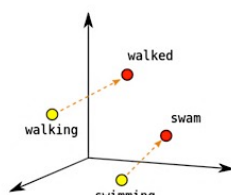
The network progressively captures the subtle meanings of each word in the vocabulary, bringing dense language to life



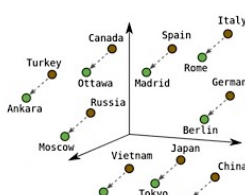
Word2Vec CBOW Embeddings: Relationships and Analogies



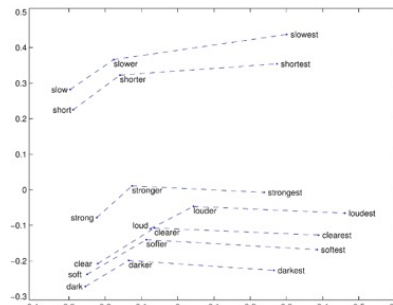
Male-Female



Verb-Tense



Country-Capital

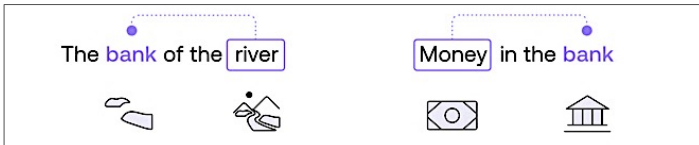


Adjective Types

- Semantic and linguistic relationships**
3D Word2Vec CBOW embeddings after training on a 6 billion words' corpus
 - Regular and superlative adjectives** (e.g., *small, smaller, smallest* have the same angles among them as *short, shorter, shortest*)
 - Gender relationships** (e.g., *man::woman as king::?*; $\text{vector}(\text{woman}) - \text{vector}(\text{man}) + \text{vector}(\text{king}) \approx \text{vector}(\text{queen})$)
- the example of 2D Word2Vec CBOW are embeddings from IMDb reviews.

Word2Vec Shortcoming: **Monosemic Token Embeddings**

- One token, multiple meanings
 - A huge obstacle for LMs is when a token can be used in different contexts
 - When this problem is encountered, the model needs to use the sequence context to decipher which meaning of the token to use



How would a model know that the word “bank” in the first sentence refers to a setting in nature, and in the second sentence to a financial setting ?

- Word2Vec, GloVe etc. generates a single, fixed vector representation for each token in its vocabulary, regardless of the context in which the token appears
 - Once trained, a token embedding remains the same across all uses of that token
 - **Polisemy**: a person's **mouth**; the **mouth** of a river (where it meets the sea), **both indicate an opening**
 - **Homography**: the **lead** is heavy (metal) - **Lead** the way (to guide), **no shared meaning root**



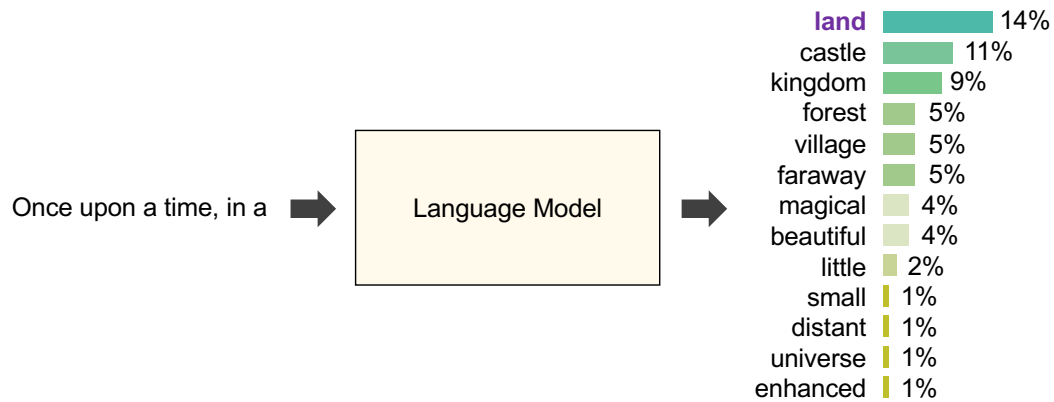
Recurrent Neural Networks

Prompt: A robot placing in sequence cards with words

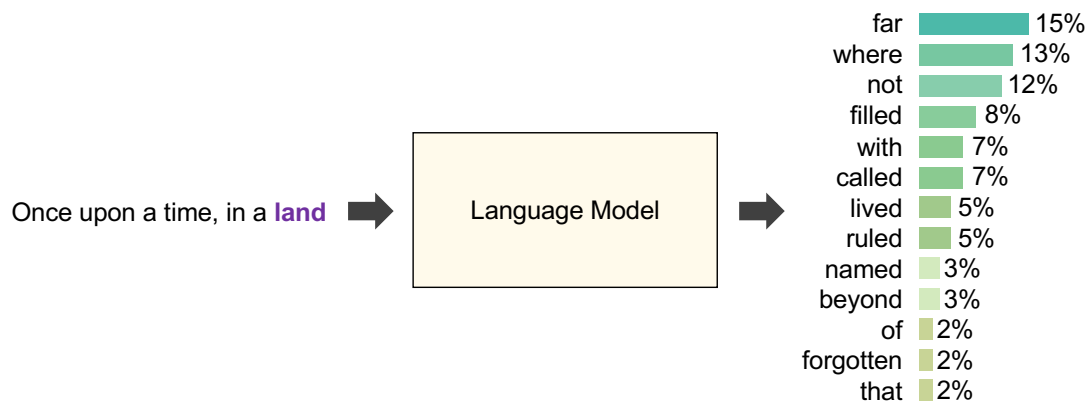
Generated with Microsoft Copilot Designer

From Representation Learning to Generative NLP

- Autoregressive text generation is a multi-classification problem!
 - Predict the most probable next token over a vocabulary, i.e. **each token is a class**

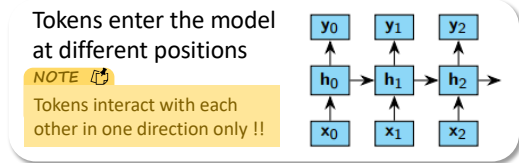


Generative NLP: Predict, Sample, Repeat



Recurrent Neural Network: First Generative Language Model

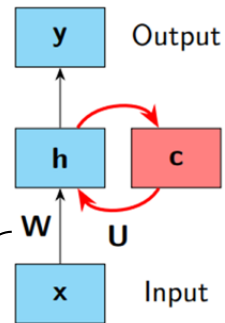
- Input data is given as a **sequence** ($x_0, x_1, \dots$) of vectors, one at each time step
- The NN responds with an output sequence ($y_0, y_1, \dots$) of the same length
- Through **cyclic connections**, the output of hidden layer(s) at **time $t-1$ is given in input at time t** , allowing to maintain a state
 - The **state** is reset at each new input sequence
- An RNN can be trained like a deep NN with one layer for each time step (*Backpropagation Through Time - BPTT*)



NOTE

- A simple addition handles state across layers
- State injection may not always be necessary

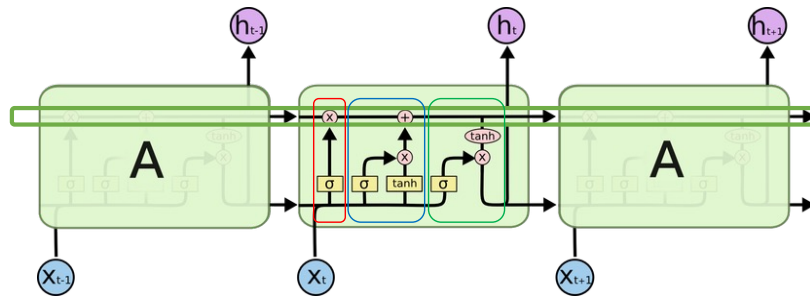
W , U and b are Trainable parameters



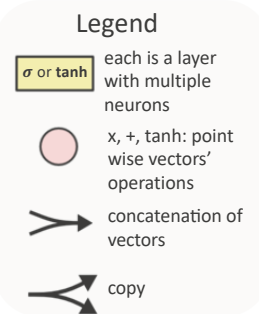
RNNs: Long Short Term Memory Networks

- Training a RNN is particularly challenging
 - Input data can have uncontrolled effects on the RNN internal state
 - During backpropagation, the **gradient** values at later time steps can become negligible (or prevalent) when evaluated at earlier ones
 - This problem is known as **vanishing** (or **exploding**) *gradient*
 - Incapacity to equally **correlate** all input vectors, mostly at **distant time steps**
- Long Short Term Memory** is a well-known approach to overcome these problems [[Hochreiter and Schmidhuber, 1997](#)]
 - A RNN can have more **layers of LSTM cells**, each storing a value vector
 - The information flow across each cell is controlled by **gates**, which can (partially) prevent the memory from being written or read
 - State is retained** “by default” across time steps, allowing to the correlation of distant inputs and preventing gradient from vanishing or exploding
 - Widely used in NLP, image and video captioning, speech recognition ...

LSTM Cell is Composed by 4 Layers

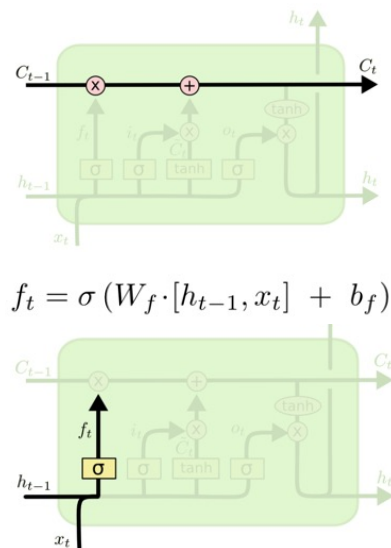


- The diagram shows the same cell at 3 subsequent time steps $t-1$, t and $t+1$
- Layer 1: the upper line contains the **memory state**
- Layer 2: **forget gate** controls whether state is retained
- Layer 3: **input gate** controls whether state is written from input
- Layer 4: **output gate** controls whether state is emitted



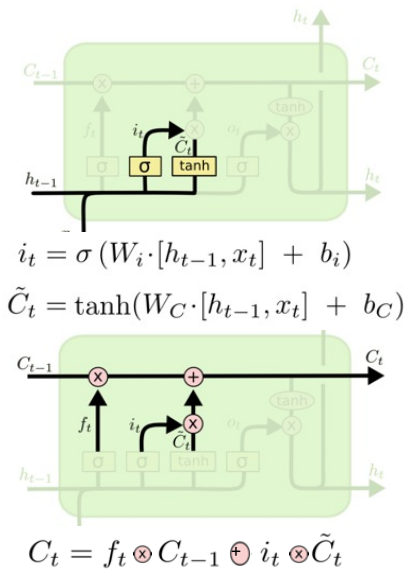
LSTM Cell: *Forget Gate* f_t

- Cell state C_t is like a conveyor belt of information depending from previous C_{t-1}
 - and from interactions with other internal layers
- The *forget gate* f_t learns how much of previous C_{t-1} should be forgotten:
 - f_t output is a vector of real values between 0 and 1 which is point wise multiplied $\otimes$ by C_{t-1}
 - multiplication by 1, or close to 1, preserves completely a value and 0 deletes/reduces it, like a tap
 - f_t is a logistic regression: i.e. it learns a hyper-plane $W_f \cdot [h_{t-1}, x_t] + b_f$ and $\sigma() \in [0, 1]$
 - W_f and b_f are the parameters to be learned and $[h_{t-1}, x_t]$ is the concatenation of previous hidden state h_{t-1} with the current input data x_t



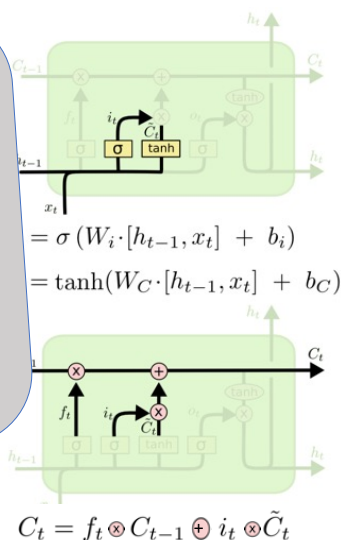
LSTM Cell: *Input Gate*

- *input gate* learns what new information to store in the cell state C_t as follows:
 - $\hat{C}_t$ is the new information derived by learning a hyperplane $W_C \cdot [h_{t-1}, x_t] + b_C$ with parameters W_C and b_C from the same data $[h_{t-1}, x_t]$
 - the resulting vector elements are squished in $[-1, 1]$ with $\tanh()$ to avoid the vanishing/exploding gradient
 - input gate i_t learns with a sigmoid (aka logistic regression) another hyperplane to weight $\hat{C}_t$
 - with parameters W_i and b_i , which rules how much of new information $\hat{C}_t$ is to be kept by the point wise multiplication $i_t \otimes \hat{C}_t$
- the new cell state C_t is derived by a point wise weighted sum of C_{t-1} and $\hat{C}_t$
 - C_{t-1} is weighted by forget gate f_t and $\hat{C}_t$ by input gate i_t



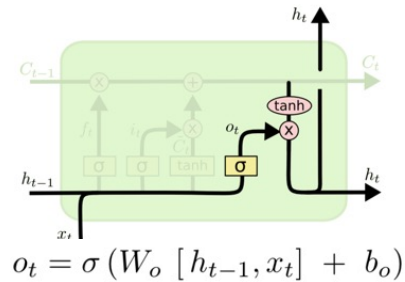
LSTM Cell: *Input Gate* - Example in NLP

- *input gate* learns what new information to store in the cell state
 - E.g. the net is *learning* a language model to *predict* the *next word* based on the *previous sequence*
 - E.g. "the *girl* said ... *she* would come"
 - to *predict* a *pronoun*, it *learned* with *input gate* i_{t-1} to *remember* in the cell *state* C_{t-1} the *subject gender* previously seen
 - When a *new subject* arises, it also *learns* the new cell *state* C_t to *forget* with f_t gate the *gender* of the old *subject*
 - E.g. "the *boy* mentioned ... *he* was ready"
- the new cell state C_t is derived by a point wise weighted sum of C_{t-1} and $\hat{C}_t$
 - C_{t-1} weighted by forget gate f_t and $\hat{C}_t$ by input gate i_t



LSTM Cell: *Output Gate*

- *output gate* rules what part of cell state C_t to emit in h_t
 - firstly $\tanh(C_t)$ squishes C_t in $[-1, 1]$ to avoid the vanishing/exploding gradient
 - output gate o_t learns with a sigmoid a hyperplane $W_o \cdot [h_{t-1}, x_t] + b_o$ with parameters W_o and b_o from the $[h_{t-1}, x_t]$
 - again a sigmoid rules what part of $\tanh(C_t)$ is emitted in h_t multiplying o_t with $\tanh(C_t)$
- h_t in the cell top is the LSTM cell output
 - i.e. h_t in the top propagates to the next network layer, whatever type it is
 - moreover h_t in the cell bottom is the recurrent link



Note: for each gate g , the net learns a different hyperplane with distinct parameters W, b
 $W_g \cdot [h_{t-1}, x_t] + b_g$

GRU: Variants and Evolutions of LSTM

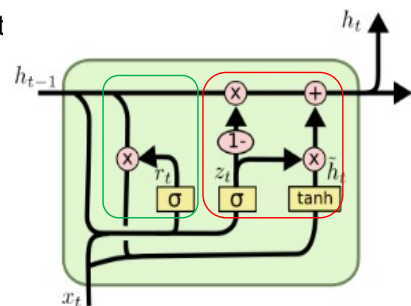
- There are several variants over the basic LSTM model
 - forget gate may be absent, *peephole* connections may be used to feed the cell memory state to all gates (thousands of variations tested ...)
- *Gated Recurrent Units (GRU)* are a major LSTM variation reducing the number of parameters [Cho, et al. (2014)]
 - memory and hidden state are merged
 - GRU only uses an *update gate* (combining input and forget (controlling state output))
 - GRU are more efficient than LSTM
 - LSTM may requires larger dataset to get higher accuracy than GRU

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$



LSTM: An Example in NLP – i

- An LSTM trained on Leo Tolstoy's War and Peace to generate samples every 100 iterations of training. After 100 iterations, the model samples seem **random** ...

```
tyntd-iafhatawiaoihrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e  
plia tkrlrgd t o idoe ns,smtt h ne etie h,hregtrs nigtike,aoaenns lng
```

- But, it **starts to separate words by spaces**, even if sometimes it puts 2 spaces. It *ingores that a comma is almost always followed by a space*. At 300 iterations the model starts to use **quotes and periods at the end of sentence**

```
"Tmont thithey" fomesscerliund  
Keushey. Thom here  
sheulke, anmerenith ol sivh I lalterthend Bleipile shuwly fil on aseterlome  
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."
```

- At iteration 500, the model has learned to spell the shortest and most **common words** such as *"we"*, *"He"*, *"His"*, *"Which"*, *"and"*, etc.

```
we counter. He stutn co des. His stanted out one ofler that concoissions and was  
to gearang reay Jotrets and with fre colt off paitt thin wall. Which das stimn
```

Source: <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

[disi-unibo-nlp.github.io](https://github.com/disi-unibo-nlp)

Prelude to Transformers - Moro G., Frisoni G.

52

LSTM: An Example in NLP – ii

- At iteration 700, we start to see more and more **English-like text**

```
Aftair fall unsuch that the hall for Prince Velzonski's that me of  
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort  
how, and Gogition is so overelical and offer.
```

- At iteration 1200, we're now seeing the use of quotations and **question/exclamation** marks. **Longer words** have now been learned as well

```
"Kite vouch!" he repeated by her  
door. "But I would be done and quarts, feeling, then, son is people...."
```

- Eventually, we begin to see **correctly spelled** words, quotations, names, and more around iteration 2000

```
"Why do what that day," replied Natasha, and wishing to himself the fact the  
princess, Princess Mary was easier, fed in had oftended him.  
Pierre aking his soul came to the packs and drove up his father-in-law women.
```

[disi-unibo-nlp.github.io](https://github.com/disi-unibo-nlp)

Prelude to Transformers - Moro G., Frisoni G.

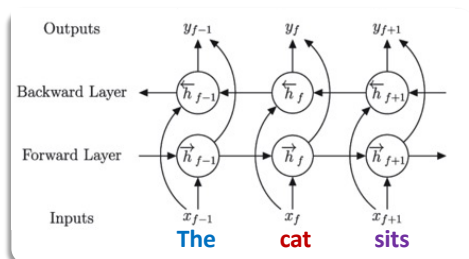
53

LSTM: an Example - Summary (iii)

- the model first discovers the general word-space structure and then rapidly starts to learn the words;
- First starting with the short words and then eventually the longer ones.
- Topics and themes that span multiple words (and in general longer-term dependencies) start to emerge only much later

Bidirectional LSTM: State-of-the-art in NLP Up To 2017

- The principle is to split RNN neurons into **two directions**
 - one for positive time direction (forward states) and another for negative time direction (backward states)
 - The forward layers read the input from the first token while the backward from the last one
- By using two-time directions, input information from the past and future of the current time frame are then combined to generate the output
 - unlike standard RNN, which requires delays to include future information



- In the forward layer **cat** is influenced by **The**
- In the backward **cat** is influenced by **sits**

NOTE

- BRNNs are one of the first attempts to process information in different directions effectively
- Bidirectionality allows for context-intensive tasks such as long document summarization, extractive Question-Answering, etc.



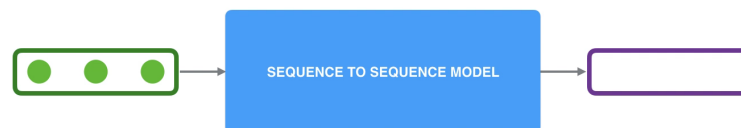
Seq2Seq

Prompt: Transformer toy looking at the horizon and the origin of a new universe

Generated with Microsoft Copilot Designer

Seq2Seq: Introduction

- Sequence-to-sequence (seq2seq) models
 - A seq2seq model **takes a sequence of items** (e.g., words, letters, image features) and **outputs another sequence of items**
 - A seq2seq model **allows inputs and outputs of different sizes**
- Successful in generative tasks such as Machine Translation (*driving field*), Text Summarization and Image Captioning
 - 2 pioneering papers [[Sutskever et al., 2014](#), [Cho et al., 2014](#)]; a revelation at the time
 - Google Translate started using them in production in late 2016 [[Source](#)]



Seq2Seq: Inference and Training

- Formally, each seq2seq model transforms an input sequence $x = (x_1, x_2, \dots, x_m)$ into an output sequence $y = (y_1, y_2, \dots, y_n)$ where n, m can be different
- Inference:
 - The seq2seq model, with parameters θ , outputs the sequence y given the input sequence x that maximizes the probability chain $P_\theta(y|x)$
$$P_\theta(y|x) = \prod_{t=1}^n P_\theta(y_t|y_1, y_2, \dots, y_{t-1}, x)$$
 - Each step t depends from the context $y_1, y_2, \dots, y_{t-1}$ and the complete input x
 - In greedy decoding, each step t selects the most probable token $y_t = \arg \max_{y_t} P_\theta(y_t|y_1, \dots, y_{t-1}, x)$
- Training
 - learning the best possible parameters θ^* that maximize the associations among the instances x, y of the dataset $\theta^* = \operatorname{argmax}_{\theta} P(y|x)$

Seq2Seq: Inference and Training – Beam Search Decoding

- With Beam Search, the model maintains B of the most promising sequence paths (where B is the beam width) rather than one
 - multiple paths, increasing the chance of finding a more optimal sequence by exploring multiple hypotheses
- A translation example with $B = 3$
 - Input:** "The cat is on the mat."
 - at each step, the model retains the top 3 partial sequences
 - 1. ["Il", "Un", "Quel"]
 - 2. ["Il gatto", "Un gatto", "Quel gatto"]
 - 3. ["Il gatto è", "Un gatto è", "Quel gatto è"]
 - 4. ["Il gatto è sul tappeto", "Un gatto è sul tappeto", "Quel gatto è sul tappeto"]
- Selection of the sequence with the max probability: "Il gatto è sul tappeto."

Seq2Seq: Generate a Sequence w.r.t. Another One

▪ Conditional language models (CLMs)

- While decoder only LMs estimate the unconditional probability $p(y)$ of a sequence y , seq2seq models need to estimate the conditional probability $P(y|x)$ given a source x
- That's why seq2seq tasks can be modeled as CLMs: they operate similarly to LMs, but additionally receive source information

Decoder only LM that starts from a prompt

$$P(y_1, y_2, \dots, y_n) = \prod_{t=1}^n p(y_t | y_{<t})$$

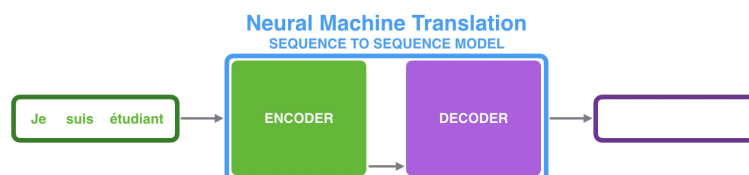
Conditional Language Models,
i.e. encoder-decoder:

$$P(y_1, y_2, \dots, y_n | x) = \prod_{t=1}^n p(y_t | y_{<t}, x)$$

Note: Conditional language modeling is something more than just a way to solve seq2seq tasks. In the most general sense, x can be something other than a sequence of tokens. For example, in the Image Captioning task, x is an image and y is a description of this image.

Seq2Seq Architecture: Encoder and Decoder

- A seq2seq model is composed of an **encoder** and an autoregressive **decoder**
- The encoder processes the input sequence one step at a time and outputs the information it captures into a **context** vector
 - After processing the entire input sequence, the **encoder sends a context vector to the decoder**, which begins producing the output sequence, item by item
 - Note: the **context** vector has a **fixed size** independently from the input length
 - The encoder and the decoder implemented with **recurrent neural networks (RNNs)** reduce the vanishing gradient problem using
 - Long Short-Term Memories (LSTMs) and Gated Recurrent Units (GRUs)



Seq2Seq Architecture: Encoder-Decoder Origin

- A seq2seq model is composed of an **encoder** and an autoregressive **decoder**



Clear role distinction in **modeling** and **generation**

“Let’s first model a representation of the entire input sequence and then decide what and how to use the representation to generate the output”

Seq2Seq Architecture: Generate only after all Input Reading

- A seq2seq model is composed of an **encoder** and an autoregressive **decoder**
 - The encoder processes the input sequence one step at a time, and compiles the information it captures into a **context** vector overcoming the issue below



- **Back in the old days**, phrase-based translation by **breaking up source sentences into multiple chunks and translated them phrase-by-phrase**
- This led to **disfluency** in the translation outputs and was not quite like how we, humans, translate
- **We read the entire source sentence, understand its meaning, and then produce a translation**
- A vanilla seq2seq model mimics that!

Seq2Seq Architecture: Encoder and Decoder are RNNs

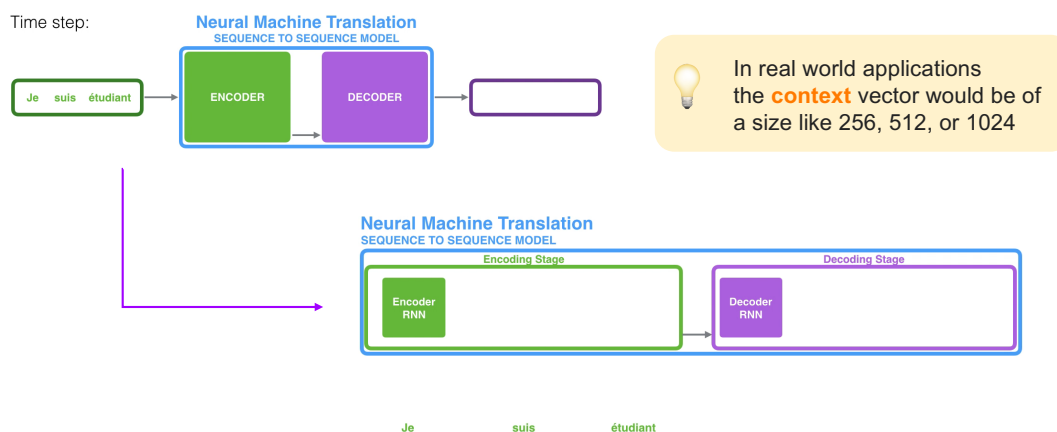
- A seq2seq model is composed of an **encoder** and an autoregressive **decoder**
 - The encoder and the decoder are usually implemented with **recurrent neural networks (RNNs)**, where LSTMs and GRUs help to avoid vanishing gradients



- A seq2seq model can have different modifications
- The RNN models differ in terms of:
 - a) **Directionality** – unidirectional or bidirectional
 - b) **Depth** – single- or multi-layer
 - c) **Type** – often either a vanilla RNN, a LSTM, or a GRU
- More information about RNNs and LSTMs on [this blog post](#)

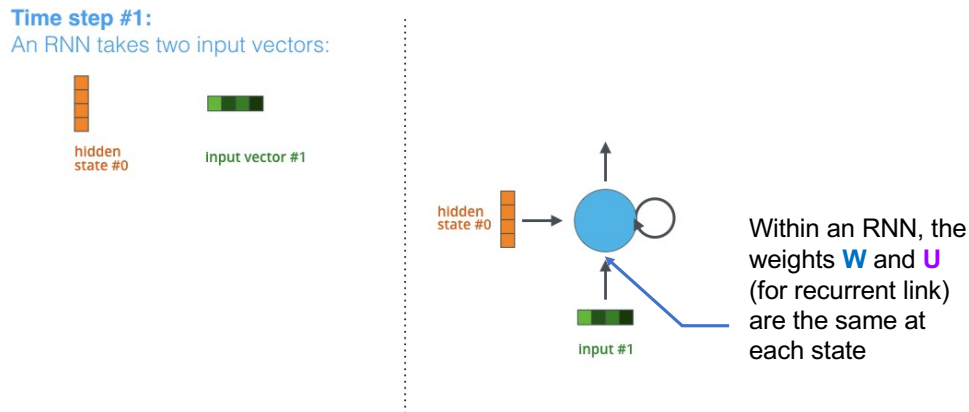
Seq2Seq Architecture: Unrolled View

- Unrolled visualization of a seq2seq model (build-then-deconstruct)



Seq2Seq Architecture: Two Distinct RNNs

- RNN mechanics: hidden states depend on an **earlier state** and an **input**

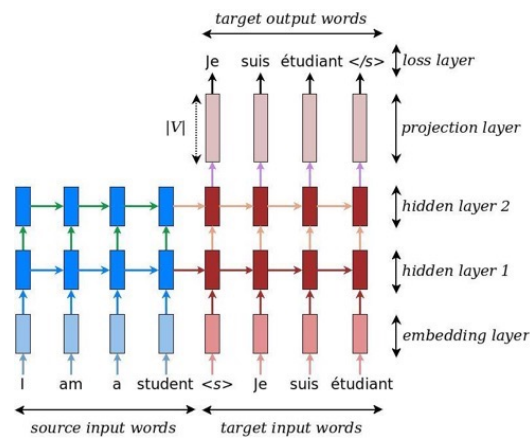


Note 1: The two RNNs in a seq2seq model, in principle, can share the same weights; however, in practice, we often use two different RNN parameters (such models do a better job when fitting large training datasets)

Note 2: the last encoder hidden state is actually the context we pass to the decoder

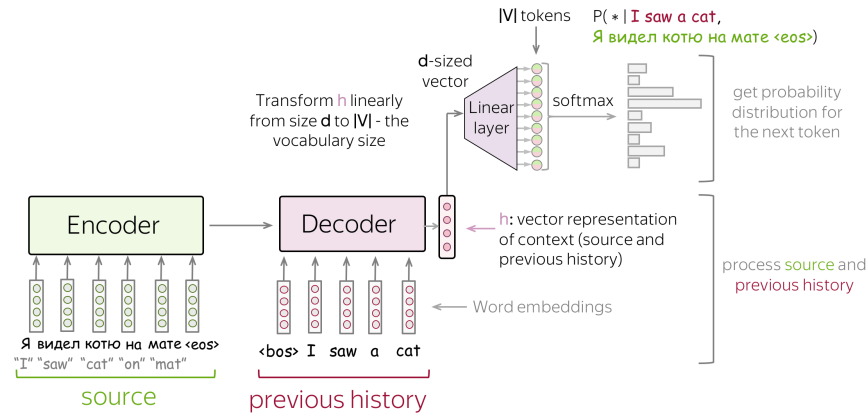
Seq2Seq Architecture: Multi-Layered RNNs

- Example of a deep multi-layer RNN-based architecture
 - Here, $\langle s \rangle$ marks the start of the decoding process, while $\langle /s \rangle$ tells the decoder to stop



Seq2Seq: Autoregressive Generation

- The modeling and training is very similar to autoregressive generative LMs
 - Get vector representation of context (both source and previous target)
 - From this vector representation, predict a probability distribution for the next token



Seq2Seq Training: Cross-Entropy Loss

- Trained to predict probability distributions of the next token, given previous context (source and previous target tokens), over the entire Vocabulary
 - Intuitively, at each step it maximizes the probability a model assigns to the correct token
 - We have a training instance with the source $x = x_1, \dots, x_m$ and the target $y = y_1, \dots, y_n$
 - At the timestep t , a model predicts a probability distribution $p^{(t)} = p(\cdot | y_1, \dots, y_{t-1}, x_1, \dots, x_m)$
 - The target at this step is $p^* = \text{one-hot}(y_t)$, i.e., we want a model to assign probability 1 to the correct token, y_t , and zero to the rest
- Cross-entropy loss for the target distribution p^* and the predicted distribution p

$$\text{Loss}(p^*, p) = -p^* \log(p) = -\sum_{i=1}^{|V|} p_i^* \log(p_i)$$

Since only one of p_i^* is non-zero (for the correct token y_t), we will get

$$\text{Loss}(p^*, p) = -\log(p_{y_t}) = -\log(p(y_t | y_{<t}, x))$$

For the whole example: $-\sum_{t=1}^n \log(p(y_t | y_{<t}, x))$

Seq2Seq Training: Illustration for a Single Timestep

Source sequence: Я видел котю на мате <eos>
 "I" "saw" "cat" "on" "mat"

Target sequence: I saw a cat on a mat <eos>
 ← one training example
 ← one step for this example

previous tokens we want the model to predict this

Model prediction: $p(* | \text{I saw a, Я ... <eos>})$

Target

Loss = $-\log(p(\text{cat})) \rightarrow \min$

decrease increase decrease

Seq2Seq Training: Complete Illustration

Encoder: read source

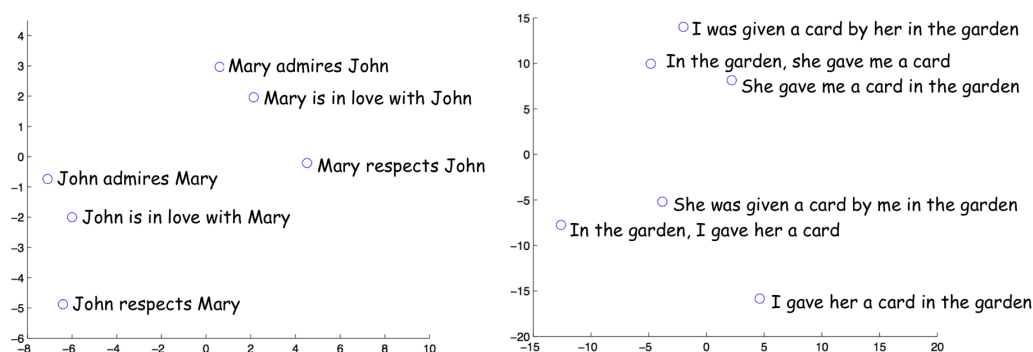
we are here

Source: Я видел котю на мате <eos>
 "I" "saw" "cat" "on" "mat"

Target: I saw a cat on a mat <eos>

Seq2Seq: Visualizing Encoder Embeddings

- [Sutskever et al. \(2014\)](#) looked at the **last encoder state** and visualized several examples: interestingly, representations of sentences with similar meaning but different structure are close



Seq2Seq Pseudocode: Encoder

- Typically, the encoder RNN uses a zero vector as starting state
- Dynamic RNNs allow for variable sequence lengths
- `encoder_emb_inp` represents the input after the embedding layer
[`max_time`, `batch_size`, `num_units`]; `encoder_outputs` are ignored

```
# Build RNN cell
encoder_cell = LSTMCell(num_units)

# Run Dynamic RNN
# encoder_outputs: [max_time, batch_size, num_units]
# encoder_state: [batch_size, num_units]
encoder_outputs, encoder_state = dynamic_rnn(
    cell=encoder_cell,
    inputs=encoder_emb_inp,
    sequence_length=max_time)
```

Single-layer LSTM encoder inspired from [\[Luong et al., 2017\]](#)

Seq2Seq Pseudocode: Decoder

- The decoder also needs to have access to the source information
 - A simple way: initialize it with the last hidden state of the encoder, `encoder_state`

```
# Build RNN cell
decoder_cell = LSTMCell(num_units)

# Dense matrix to turn the top hidden states to
# logit vectors of dimension V
projection_layer = Dense(
    tgt_vocab_size, use_bias=False)

# Decoder
decoder = Decoder(
    cell=decoder_cell,
    sampler=TrainingHelper(decoder_emb_inp),
    initial_state=encoder_state,
    output_layer=projection_layer)
```

Single-layer LSTM decoder inspired from [\[Luong et al., 2017\]](#)

Seq2Seq Pseudocode: Loss

- `logits`
 - Unnormalized output from decoder's final layer
- `maximum_iterations`
 - Limit the translation length since we do not know the output sequence length in advance
- `decoder_targets`
[`max_time`, `batch_size`]
 - Target output tokens, i.e., decoder inputs shifted to the left by one time step with an end-of-seq tag appended on the right
- `target_weights`
 - Zero-one matrix of the same size as `decoder_targets`; it masks padding positions outside the target sequence lengths with 0 values

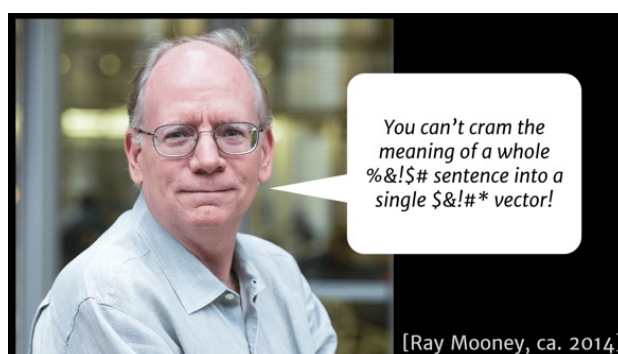
```
# Dynamic decoding
logits = dynamic_decode(
    decoder, maximum_iterations)

# Cross-entropy loss
crossent = sparse_softmax_cross_entropy_with_logits(
    labels=decoder_targets, logits)
train_loss = reduce_sum(crossent * target_weights) /
    batch_size
```

Information Bottleneck

▪ The problem of fixed-size context vector (memory)

- (i) for the encoder, it is **hard to compress all the information** from the sequence; the number of possible source sequences (hence, their meanings) is $\infty \rightarrow$ it is likely to forget something
- (ii) for the decoder, **different information may be relevant at different steps**
- As sequence size increases, model performance slumps (also with LSTMs and GRUs)



Let's Pay **Attention**: Give Some Inputs More Weight

- 💡 Break down the **input text** representation into a **sequence of hidden states**...
- We could **consider each token vector with its individual information** instead of **collapsing them all into one big vector**
- ... But such a solution still has obvious flaws with memory and context

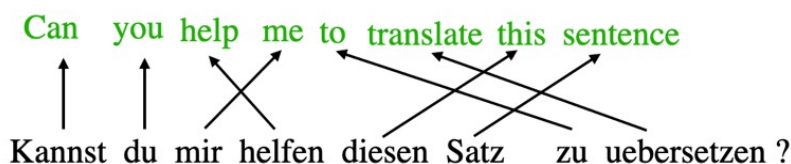


How to build a time and memory efficient model that predicts accurately from a long sequence ?

Maintaining all token hidden states might work if the model had a way of **selecting and focusing on the likeliest input tokens** each step

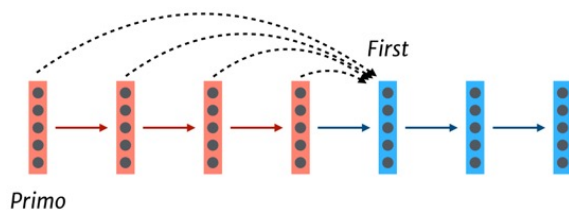
Seq2Seq + Attention: A Machine Translation Example

- For instance, consider translating a sentence from one language to another
 - Translating each token only based on the context vector does not work effectively
 - The attention was introduced to **give access to all sequence items at each time step**
 - The key is to **be selective and determine which tokens are important in a specific context**

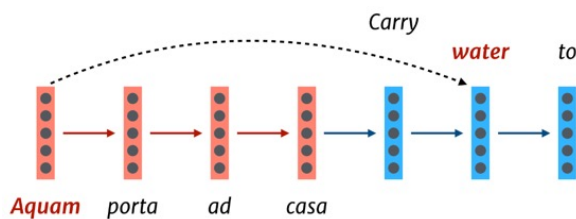


Seq2Seq + Attention: Learnable Token Alignments

- During **decoding**, learn from what matters, ignore what doesn't
 - Token alignments cannot be hard-coded: they must be learned
 - "Alignments" is an intuitive term from statistical machine translation



We don't know which word is relevant when we add connections



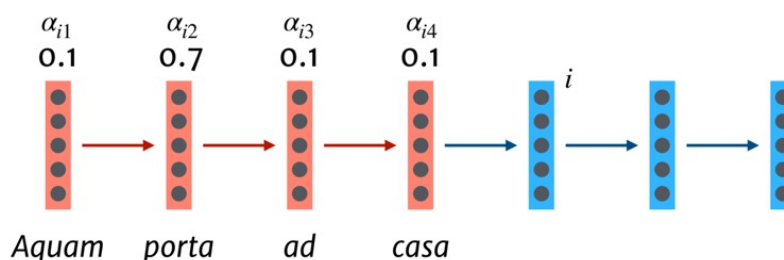
Words aren't one-to-one, and order can change

Seq2Seq + Attention: More Data, More Decoder Operations

-  Not all input tokens are equally important for a certain output token; some deserve more attention than others
- The **attention mechanism** is ubiquitous in modern deep learning models
 - Introduced and refined by [Bahdanau et al. 2014; Luong et al., 2015] for machine translation
-  Add learnable attention layers to help the model **understand the weight of each input token at each decoder step** and as needed
 - The attention mechanism is part of a neural network and **allows the decoder to «look back»** at the complete input and **retrieve helpful information**
 - The ability to **amplify the signal from the relevant part of the input sequence** makes the attention models produce better results than models without attention
- In summary, at each time step...
 - The **encoder** passes a lot more data to the **decoder** (i.e., all the RNN hidden states instead of the last one)
 - An **attention-empowered decoder** does extra operations before producing its output

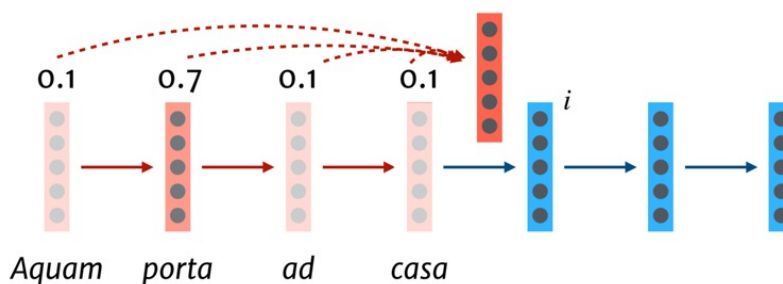
Seq2Seq + Attention: The Decoder Operations – i

1. When predicting output i , assign a weight α_{ij} to each encoder state h_j (weights sum to 1)



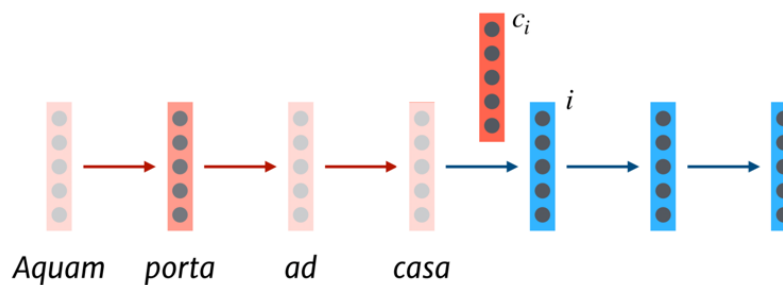
Seq2Seq + Attention: The Decoder Operations – ii

1. When predicting output i , assign a weight α_{ij} to each encoder state h_j (weights sum to 1)
2. Compute a pooled input $c_i = \sum_j \alpha_{ij} h_j$ («attention output» or «context vector»)



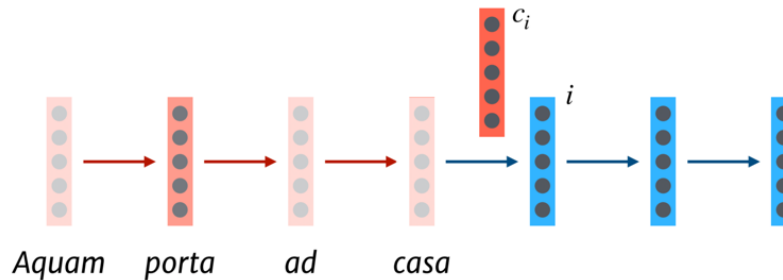
Seq2Seq + Attention: The Decoder Operations – iii

1. When predicting output i -th, assign a weight α_{ij} to each encoder state h_j (weights sum to 1)
2. Compute a pooled input $c_i = \sum_j \alpha_{ij} h_j$ («attention output» or «context vector»)
3. Use c_i to update the decoder parameters for its output i -th



Seq2Seq + Attention: The Decoder Operations – iv

1. When predicting output i -th, assign a weight α_{ij} to each encoder state h_j (weights sum to 1) ... *How?*
2. Compute a pooled input $c_i = \sum_j \alpha_{ij} h_j$ («attention output» or «context vector»)
3. Use c_i to condition the decoder output i -th ... *How?*



Seq2Seq + Attention: How to Compute Attention Weights

- Three popular ways to implement the attention score function

- h_t = decoder state at step t ; s_k = encoder state for token k

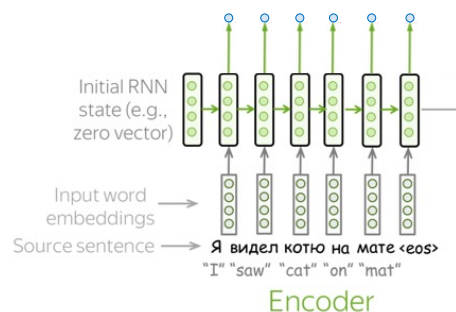
Dot-product	Multi-Layer Perceptron	Bilinear
$\text{score}(h_t, s_k) = h_t^T s_k$	$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1 [h_t, s_k])$	$\text{score}(h_t, s_k) = h_t^T W s_k$
Element-wise Multiplication The simplest method <i>(computational efficient but limited in expressiveness)</i>	Concatenation + Feed-forward layer w/ tanh activation function W_1 + weight vector w_2 «Bahdanau attention» [Bahdanau et al., 2014], the paper that presented the attention for the first time, co-authored by Yoshua Bengio <i>(expressive but computational expensive)</i>	«Luong attention» [Luong et al., 2015] Compared to dot-product, it adds a weight matrix W that allows learning more complex relations <i>(middle ground)</i>

Seq2Seq + Attention: How to Use the Attention Output

- The attention output c_i is combined with the current decoder state to produce an **attentional hidden state**
 - This is usually done through **concatenation followed by a linear transformation**
 - E.g., attentional state = $\tanh(W[c_i, h_t])$
- The attentional hidden state is then used to predict the output token for the current time step, typically through a softmax layer
- The predicted output is compared to the actual target output to compute the cross-entropy loss
- The loss is propagated through the network for parameters update, including:
 - Attention mechanism (W_1 and w_2 in Bahdanau, W in Luong)
 - Decoder
 - Encoder
- Key point: the attention mechanism becomes an integral part of the network

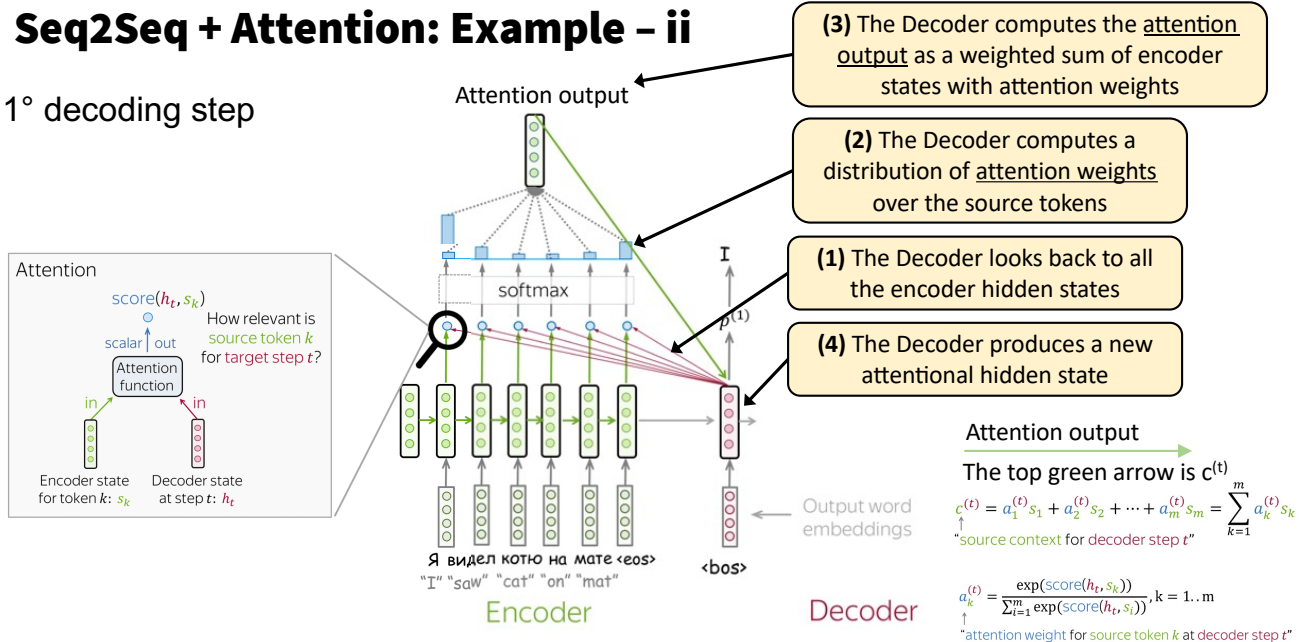
Seq2Seq + Attention: Example – i

- Full encoded input



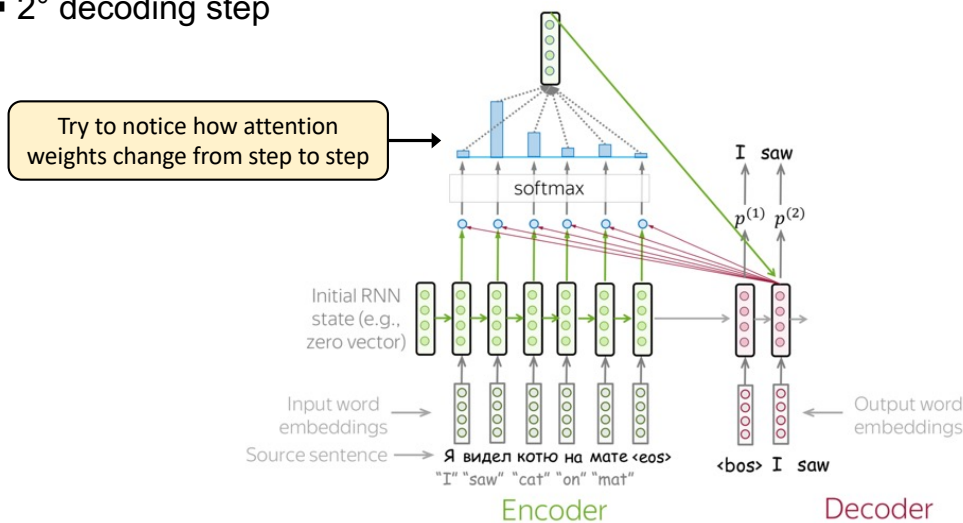
Seq2Seq + Attention: Example – ii

1° decoding step



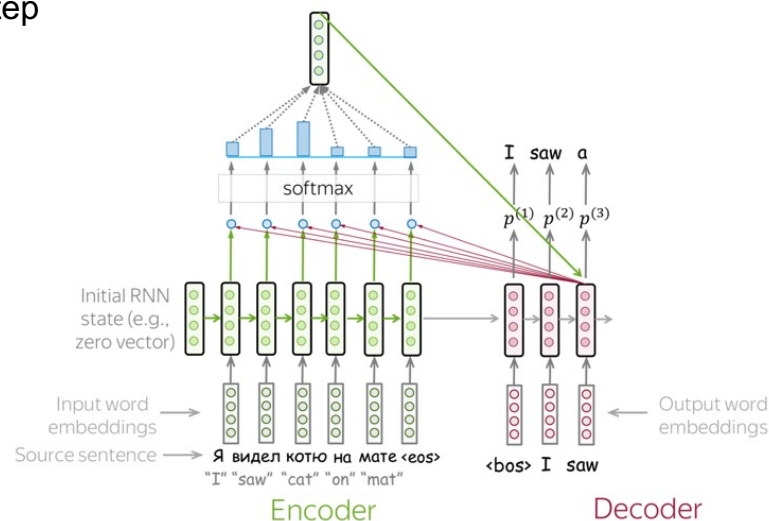
Seq2Seq + Attention: Example – iii

2° decoding step



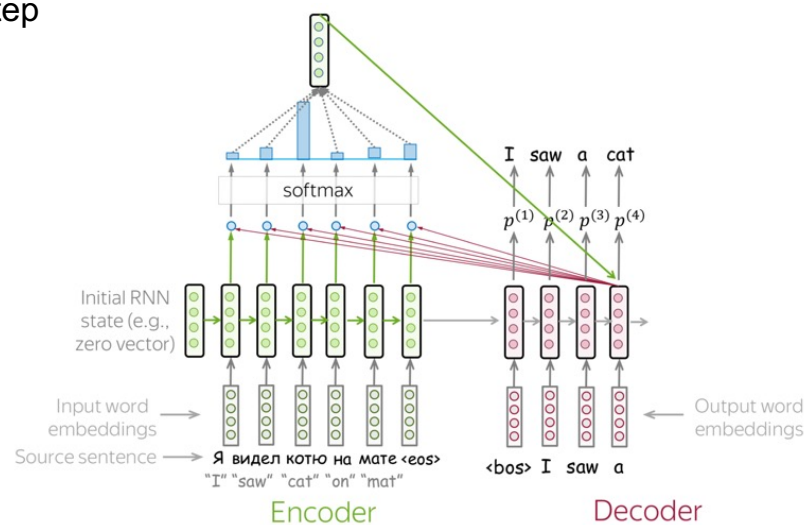
Seq2Seq + Attention: Example – iv

- 3° decoding step



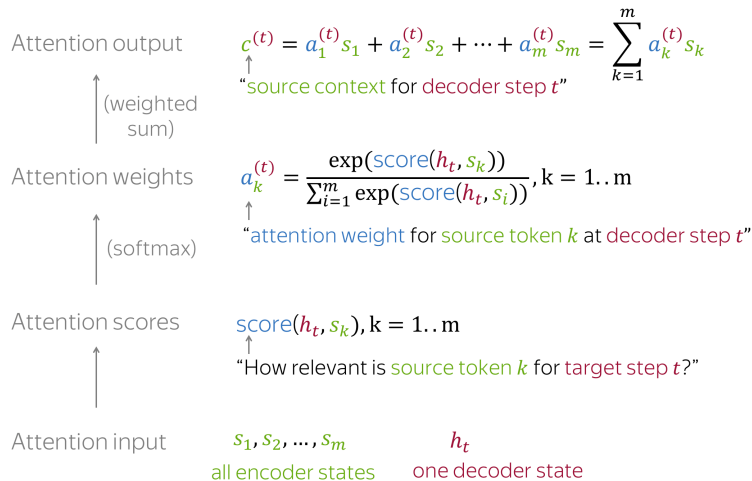
Seq2Seq + Attention: Example – v

- 4° decoding step



Seq2Seq + Attention: A Fully Differentiable Mechanism

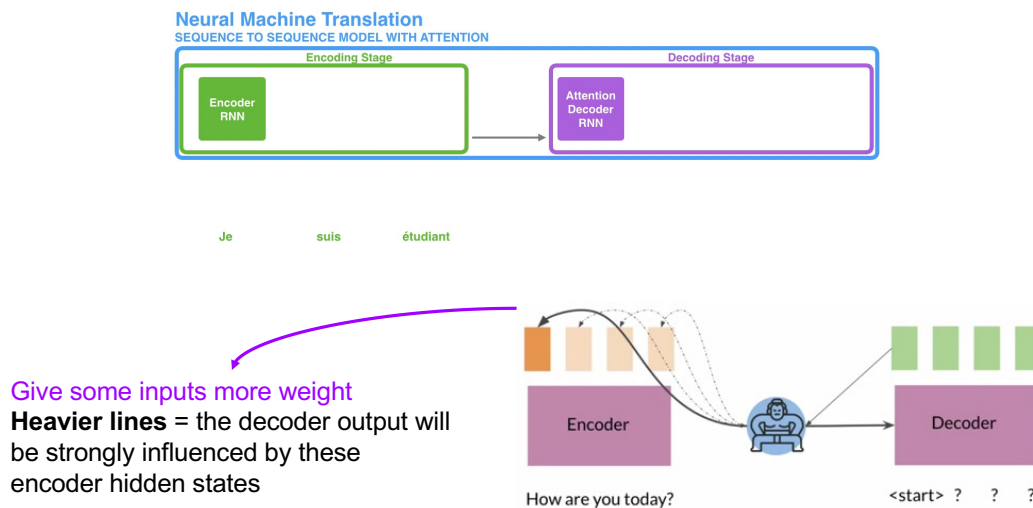
General computation scheme



Note

- Since **everything is differentiable** (attention function, softmax, etc.), a model with attention can be trained **end-to-end**
- You don't need to specifically teach the model to pick the words you want – **the model itself will learn to pick important information**
- Hard attention (argmax) would be non-differentiable!

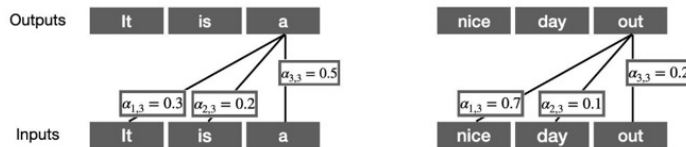
Seq2Seq + Attention: Unrolled View



Attention vs Fully-Connected Layers: A Common Question

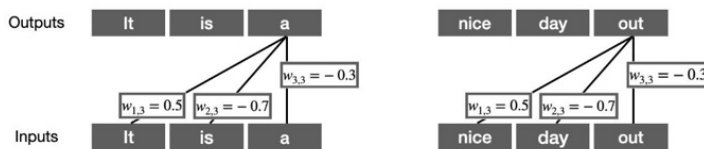
- How attention scores differ from weights in a fully-connected layer ?
 - Conceptually, attention mechanisms allow to attend to different parts of a sequence. On the surface, this sounds very similar to a fully connected layer, indeed!

Attention scores α are dynamic with respect to the input



In attention mechanisms, attention weights are **dynamic** and **input-dependent**

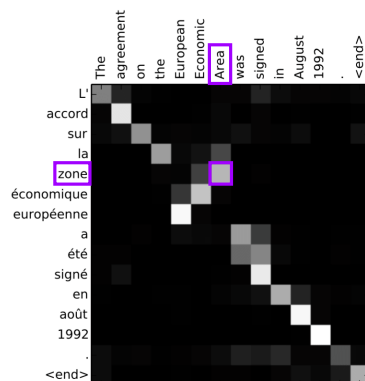
Fully-connected layer weights w are static with respect to the input



In contrast, the weights of a convolutional or fully-connected layer are **fixed after training**. As a side note: the attention can be implemented using fully-connected layers

Attention Learns (Soft) Alignments

- General and interpretable
 - Token alignments allows to better understand the context and nuances of the input data, to adapt to diverse input formats and generate more accurate predictions
 - They allow to manage different input-output token orders, grammar differences, recognize synonyms, antonyms, and other linguistic associations



The lighter squares show where the model is actually looking when translating a token; example from [Bahdanau et al., 2014]

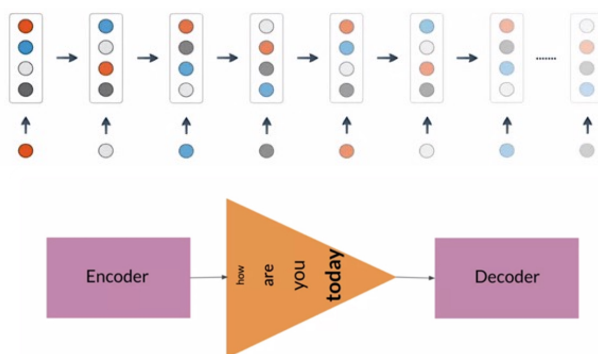
In French, "zone économique Européenne"

In English, "European Economic Area"

Attention catches the right order

Seq2Seq Shortcomings: **Decay of Information through Time**

- Even with attention, there are some flaws...
- RNNs struggles with capturing **long-range dependencies** because of their sequential nature;
 - i.e., the first input items are likely to have little effect on the distant final items



Seq2Seq Shortcomings: **Efficiency**

- Each hidden state depends on the output of the previous hidden state
- The sequential nature of the RNNs **precludes parallelization** within training examples, which becomes critical at longer sequence lengths (i.e., long processing time)
- Waiting for data to become available makes RNNs unfit with technologies like CuDNN, **slowing down the whole GPU process**
- Sequential processing also **limits batch sizes**, resulting in slower training times and increased memory requirements

Seq2Seq Shortcomings: **Monodirectional Token Encoding**

- Seq2Seq models can produce different representations for tokens depending on their context; this is achieved through the recurrent nature of RNNs
- However, traditional Seq2Seq models process tokens from left to right
 - The encoder processes the input sequence in one direction
 - At each step, it only has information about the tokens it has seen so far; it cannot incorporate information from future tokens into the representation of the current one
 - Thus, it struggle to properly embed a token when relevant context comes after it

I arrived at the **bank** after crossing the ...

What does **bank** mean in this sentence?

...street? ...river?

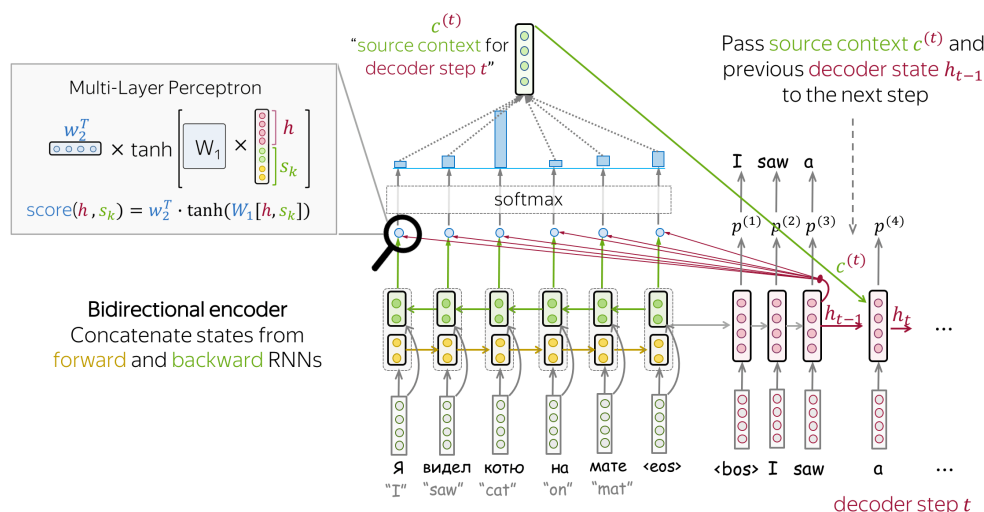


I've no idea: let's wait until I read the end

RNNs

Seq2Seq: **Bidirectional RNNs as Encoder**

- Some works use two RNNs (left-to-right, right-to-left) and combine their output

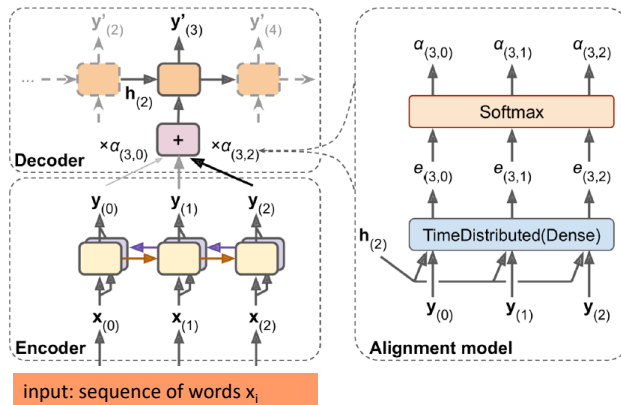


Simplified View of Attention Mechanism in Recurrent Neural Nets

It has been firstly included in recurrent neural networks based on Encoder-Decoder models for NLP

the figure depicts a bidirectional recurrent neural network where the input is supplied as a sequence of words to the encoder, **here the focus is in the 3rd step output $y'_{(3)}$**

The $y_{(i)}$ encoder output for each word is the output combination of the bidirectional RNN



The Attention consists in combining each encoder output $y_{(i)}$ (here $i=0, \dots, 2$) in a new input *alphas* for the next RNN, i.e. the decoder, applying to them an operation, i.e. the concatenation (+)

The attention learns the dependencies among words by learning the new weights *alphas* from the results $e_{(i,j)}$ achieved by combining the hidden states h of the decoder with the $y_{(i)}$ output of encoder

Concatenative and Multiplicative Attention

- weights *alphas* are computed with the softmax in the last network layer
 - $\alpha_{(t,i)} = \frac{\exp(e_{(t,i)})}{\sum_j \exp(e_{(t,j)})}$
 - $y_{(i)}$ is the encoder output vector and $h_{(t)}$ is the hidden state of the decoder
 - The operation to compute the elements $e_{(t,i)}$ determines the type of attention
 - With an input sequence of L words and L output elements of the decoder, the trainable weights are $\sim L^2$
- Concatenative Attention / Additive Attention**
 - $e_{(t,i)} = v^T \tanh(W[h_{(t)}; y_{(i)}])$
 - with v the scaling parameter and $[\cdot]$ the concatenative operation
- Multiplicative Attention**
 - $e_{(t,i)} = h_{(t)}^T W y_{(i)}$
 - It is faster and more efficient than the concatenative one in memory occupancy
 - It is implemented using functions optimized for matrix computations
 - The matrix W contains values that are learned; another attention type is the dot-product attention where W is the identity matrix without trainable parameters

xLSTM: Extended Long Short-Term Memory

[Extended LSTM \(xLSTM\)](#) introduces **Exponential Gating**, enhancing the control over memory updates and providing a more nuanced flow of information compared to traditional gating mechanisms

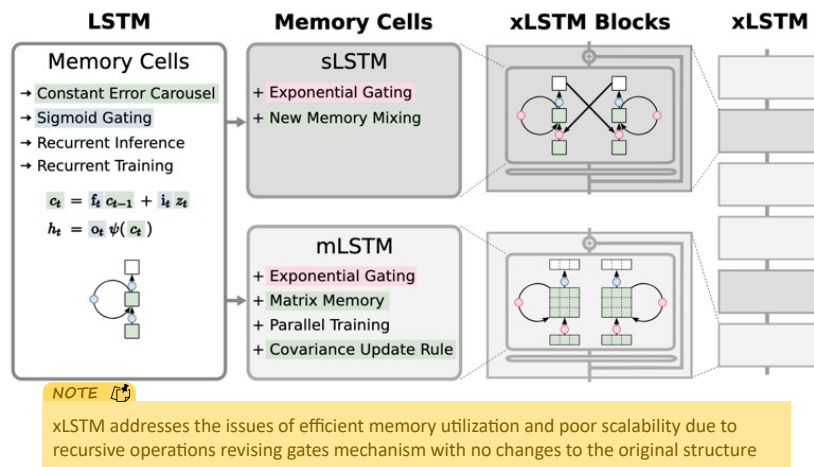
Two new memory cell mechanisms are interleaved in xLSTM.

Scalar (sLSTM)

» Higher memory usage capabilities through information stabilizer weights

Matrix (mLSTM)

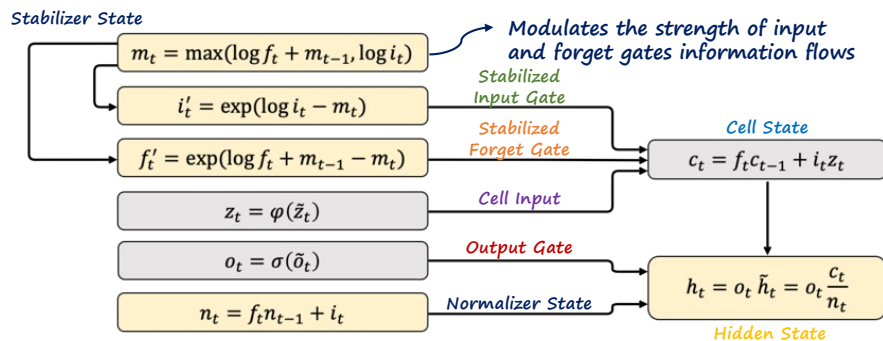
» From float to matrix memory cells
» The absence of hidden state conditioning allows for scaled parallelization



xLSTM: sLSTM cell

- Exponential gates are introduced in input and forget gates to empower LSTMs with the ability to **revise storage decisions**

- Exponential activation functions can cause overflow issues, which is why we must stabilize the gates by incorporating **two extra states** m_t and n_t



NOTE Remember this Multi-Head mechanism for future lectures...

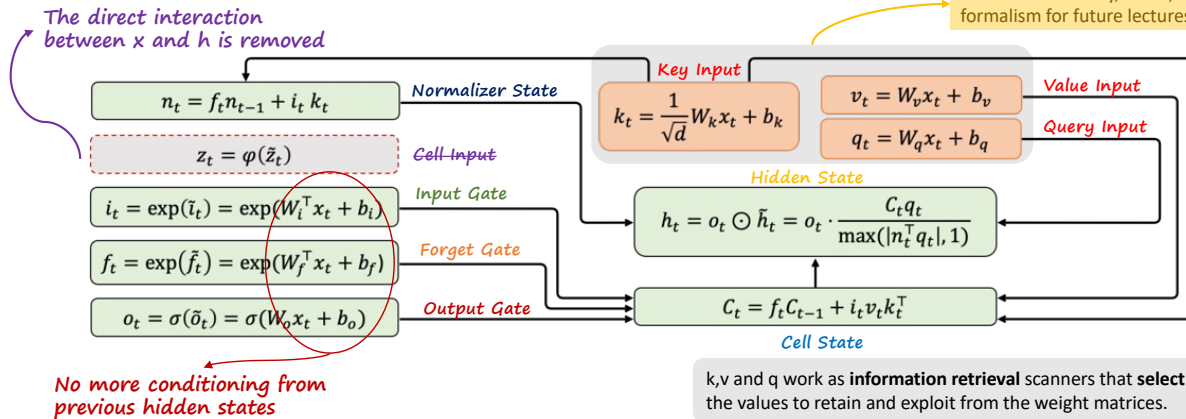
- sLSTM can be organized into **multiple parallel memory cells**
- Multiple memory heads enable memory mixing through recurrent weight connections $R_\alpha = [r_\alpha^i]$, capturing different memory interaction systems
- sLSTM's memory heads can **mix signals within each head** but not across heads at different layers to preserve the locality of information

The memory mixing process allows better control over how past information affects the current state, enhancing long-range dependency management.

xLSTM: mLSTM cell

- To enhance the storage capacities of LSTM, the scalar memory cell $c \in \mathbb{R}$ is transformed into a **matrix** $C \in \mathbb{R}^{d \times d}$
 - Since mLSTMs have no memory mixing across different cells, therefore recurrent operations can be reformulated in **parallel**
- NOTE 📌
Remember this Key Value.

NOTE  Remember this Key, Value, Query formalism for future lectures...



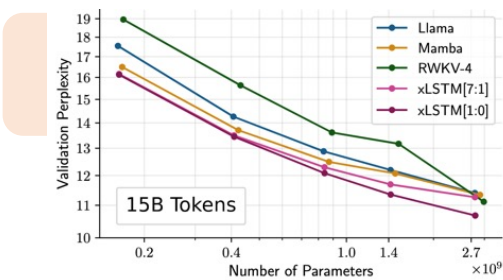
disi-unibo-nlp.github.io

Prelude to Transformers - Moro G., Frisoni G.

105

xLSTM: Scaling is NOW Possible

The more efficient information processing gates allow LSTM architecture with a higher parameter count and guarantee more fair and direct comparison with other language model paradigms

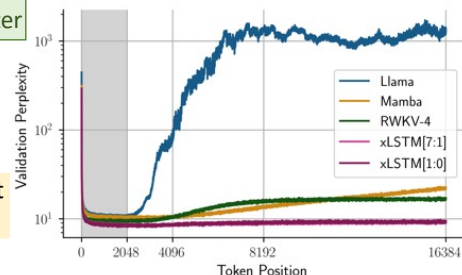


On a next-token prediction task, perplexity scales proportionally with the parameter count while retaining efficiency in comparison with Transformers, RWKV, and Mamba models after training on 15B tokens

! The lower the perplexity, the better

” The notation $\text{xLSTM}[a:b]$ indicates the ratio a/b of mLSTM-based over sLSTM-based xLSTM blocks.

Perplexity as the input length increases for models trained with context window 2048. xLSTM models remain more efficient for longer texts.



disi-unibo-nlp.github.io

Prelude to Transformers - Moro G., Frisoni G.

106

RNNs: Back to being SOTA in 2024 ?

- Original LSTMs were slow due to BPTT and couldn't parallelize training
- By **removing the hidden state** dependencies from their input, forget, and update gates, the LSTM and GRU's original authors themselves proved these architectures no longer need to BPTT and can be efficiently trained in parallel

With computational and memory requirements saving, the *mini* versions of LSTM and GRU overcome the original version in performance and costs

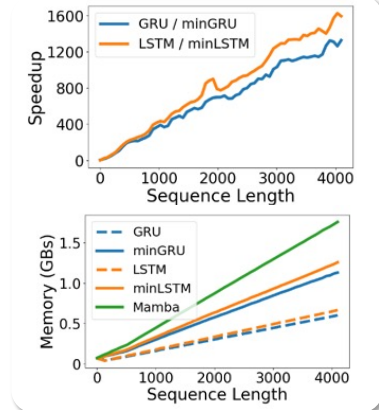
LSTM

$$\begin{aligned}
 h_t &= o_t \odot \tanh(c_t) \\
 o_t &= \sigma(\text{Linear}_{d_h}([x_t, h_{t-1}])) \\
 c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
 f_t &= \sigma(\text{Linear}_{d_h}([x_t, h_{t-1}])) \\
 i_t &= \sigma(\text{Linear}_{d_h}([x_t, h_{t-1}])) \\
 \tilde{c}_t &= \tanh(\text{Linear}_{d_h}([x_t, h_{t-1}]))
 \end{aligned}$$

⇒

minLSTM

$$\begin{aligned}
 h_t &= f'_t \odot h_{t-1} + i'_t \odot \tilde{h}_t \\
 f_t &= \sigma(\text{Linear}_{d_h}(x_t)) \\
 i_t &= \sigma(\text{Linear}_{d_h}(x_t)) \\
 \tilde{h}_t &= \text{Linear}_{d_h}(x_t) \\
 f'_t, i'_t &\leftarrow \frac{f_t}{f_t + i_t}, \frac{i_t}{f_t + i_t}
 \end{aligned}$$



disi-unibo-nlp.github.io

Prelude to Transformers - Moro G., Frisoni G.

107



Thanks for the attention

disi-unibo-nlp.github.io

Prelude to Transformers - Moro G., Frisoni G.

108